

SageJavon: A scalable AI tutor for personalized programming learning

Hanyu Zhao ^a, Yuzhuo Wu ^a, Zhufeng Lu ^a, Xiaohua Yu ^b, Weikai Miao ^a,
Liangyu Chen ^{a,*}

^a Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai, China

^b Department of Education Information Technology, East China Normal University, Shanghai, China

ARTICLE INFO

Keywords:

Programming education
Large language models
Retrieval-augmented generation
Personalized learning
Education evaluation and recommendation

ABSTRACT

Despite advancements in programming education, providing adaptive learning experiences while managing cognitive load remains a challenge. Inspired by Bloom's Taxonomy, we propose the **Learn-Practice-Evaluate-Support (LPES)** framework, integrated into SageJavon, an AI-powered tutoring system based on Large Language Models (LLMs). SageJavon overcomes the limitations of traditional AI tutors, such as rigid content delivery and a lack of personalized feedback. It includes four innovations: (1) an enhanced Retrieval-Augmented Generation (RAG) model for personalized resource delivery, (2) heuristic and follow-up questions to guide problem-solving, (3) adaptive knowledge tracing and recommendation models, and (4) the FUPS-Score for automatic code assessment. Deployed in a 12-week course with 85 students, SageJavon facilitated 5306 interactions in the Knowledge Q&A module and 5144 in the Programming Mentor module, with LLM-based scoring yielding average scores of 87.39 and 84.19. Compared to traditional tools, SageJavon reduces cognitive load, showing improved scores in physical demand, temporal demand, and performance (all $p < .001$). It also improves constructivist learning outcomes, with higher scores in prior knowledge activation (5.01 vs. 4.78), knowledge transfer (5.09 vs. 4.82), and error correction (5.24 vs. 4.68). We compare our improved RAG with other implementations and find that it outperforms the others with a 2.93% average improvement. In exercise recommendations, we achieve over 80% user satisfaction. As the number of recommended exercises increases, both SageJavon scores ($r = 0.68$) and final exam scores ($r = 0.62$) improve, demonstrating the positive impact of personalized recommendations. These results highlight SageJavon as a scalable, plug-and-play solution for programming education.

1. Introduction

Background. Since the emergence of large language models, many AI tutors within Intelligent Tutoring Systems (ITS) (Conati, 2009; Nwana, 1990) have been developed for programming education (Frankford et al., 2024; Le et al., 2013; Ma et al., 2024; Setiawan et al., 2024). Java, as one of the most widely used programming languages, plays a fundamental role in modern programming education. However, students often struggle with grasping complex programming concepts (Natalapati et al., 2024), debugging their code (McCauley et al., 2008), and developing computational thinking skills (van Haaster & Hagan, 2004). Traditional learning environments, which primarily rely on classroom instruction and textbook-based study, have not fully addressed these challenges,

* Corresponding author.

E-mail address: lychen@sei.ecnu.edu.cn (L. Chen).

<https://doi.org/10.1016/j.ipm.2025.104605>

Received 21 August 2025; Received in revised form 3 December 2025; Accepted 30 December 2025

Available online 3 February 2026

0306-4573/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

particularly in providing personalized, interactive, and adaptive learning experiences (Mota Vieira & Magalhães Netto, 2023). Several studies (Kim et al., 2026; Li et al., 2025) stress the importance of incorporating human-centered intelligent services into educational AI systems to provide more context-specific guidance for learners.

Analysis and Limitations. AI tutors designed for programming education can generally be categorized into two types: (1) *Chat-based AI tutors*, which facilitate learning through interactive conversations that help students acquire knowledge and clarify doubts; and (2) *Exercise-based AI tutors*, which reinforce learning through choice questions and programming exercises to develop problem-solving and computational thinking skills. While both approaches contribute to improving programming education, they have notable limitations in providing truly adaptive and personalized guidance. Several chat-based AI tutors have been developed for programming education (Groothuijsen et al., 2024; Yang et al., 2024), such as Python-Bot (Okonkwo & Ade-Ibijola, 2020), which assists beginners in learning Python programming, and CodeAid (Kazemitabaar et al., 2024), which provides explanations for programming concepts and algorithm-related queries. While these AI tutors can clarify programming principles and algorithms, they exhibit critical limitations: (1) They lack course-specific contextual knowledge, meaning they cannot directly address questions related to course materials such as lecture slides, textbooks, or instructor-provided resources. (2) Some AI tutors, like Python-Bot, tend to provide direct code solutions instead of guiding students through the problem-solving process, potentially diminishing students' ability to think critically and debug independently. (3) Many existing chat-based tutors lack knowledge tracing and personalization, making it difficult to assess a student's learning progress.

For exercise-based programming education (Kaila et al., 2014; Laakso et al., 2018), current programming platforms, such as online judges (OJ), mainly emphasize algorithm practice and evaluate solutions based on code execution and test cases. However, these platforms lack mechanisms for assessing students' understanding of key concepts and fail to provide personalized suggestions. While websites like LeetCode and OI Wiki organize problems by chapters,¹ allowing students to select exercises independently, they do not offer personalized exercise recommendations.

Additionally, programming education itself faces critical barriers that impede effective learning. One significant issue is cognitive overload, where students struggle to balance syntax, algorithmic logic, and debugging tasks, often exceeding their cognitive capacity (Kirschner, 2002). This is particularly problematic in Java education, where abstract concepts such as object-oriented principles (e.g., inheritance, polymorphism) require iterative understanding and problem-solving (Halford et al., 2007). Furthermore, traditional pedagogical approaches are often characterized by passive knowledge transfer, which focuses on lectures and textbooks, failing to engage students in the problem-solving process (Grigorov et al., 2017) actively. This results in missed opportunities for developing critical skills like computational thinking. Lastly, one-size-fits-all pedagogy prevalent in many programming curricula does not cater to individual learning paces and needs, limiting students' ability to master programming concepts at their own speed (Newstrom & Lengenick-Hall, 1991).

Educational Theories and Challenges. To tackle these challenges, educational theories such as *Constructivist Learning Theory* (Bada & Olusegun, 2015; Hein, 1991) and *Cognitive Load Theory* (CLT) (Kirschner, 2002) can offer valuable insights for designing more effective AI tutors. Constructivism highlights the importance of students actively constructing knowledge through interaction with their environment, while CLT emphasizes the need to optimize cognitive load in instructional designs to improve learning efficiency. However, current AI tutors often fail to incorporate these educational theories, resulting in inadequate guidance that does not fully meet students' learning needs. This gap is especially evident in programming education, where students face the dual challenge of mastering complex problem solving and debugging, both of which demand deeper conceptual understanding and personalized, adaptive support. Building on this work, Bloom's mastery learning framework further emphasizes that effective instruction should form a cyclical process of teaching, formative assessment, and corrective activities (Bloom, 1968; Forehand, 2010; Schildkamp et al., 2020). This assessment-informed instructional design aligns with the educational data mining perspective that emphasizes using learner interaction data to optimize instruction (Algarni, 2016). Inspired by these theoretical foundations and the demonstrated success of feedback-driven learning cycles in programming education (Hanks et al., 2011), we propose the Learn-Practice-Evaluate-Support (LPES) framework as a structured, iterative loop. LPES explicitly links conceptual learning, structured practice, continuous evaluation (via automated assessment and knowledge tracing), and targeted corrective support, thereby embodying a theory-informed, data-driven approach to adaptive tutoring that addresses the specific cognitive demands of learning to program.

This paper. In this paper, we propose **SageJavon**, an AI tutor designed for programming education. Built upon the LPES framework outlined above, SageJavon integrates LLMs, knowledge tracing, and advanced AI evaluation frameworks to provide a more interactive, adaptive, and personalized learning experience. LPES structures SageJavon as a repeated cycle in which learning activities are followed by practice, formative evaluation, and corrective support. The LPES framework consists of four key modules: (1) The *Learn* module for conversational learning in knowledge and programming through our improved RAG, integrating external resources and guiding students step by step. (2) The *Practice* module for exercises and behavior modeling. (3) The *Evaluate* module for assessing exercises using knowledge tracing and evaluating code automatically using our FUPS-Score evaluation framework. (4) The *Support* module for recommending personalized resources and feedback based on the student's progress. By incorporating a feedback loop where *Learn*, *Practice*, *Evaluate*, and *Support* continuously interact, SageJavon can significantly enhance the learning experience for Java programming students.

Contributions. The main contributions of our work are as follows.

1. SageJavon: An AI-Powered Tutor for Programming Education. We introduce SageJavon, an AI-driven tutoring system utilizing the "Learn-Practice-Evaluate-Support" framework to provide personalized, adaptive learning support for programming education.

¹ LeetCode: <https://leetcode.com>, OI Wiki: <https://oi-wiki.org>

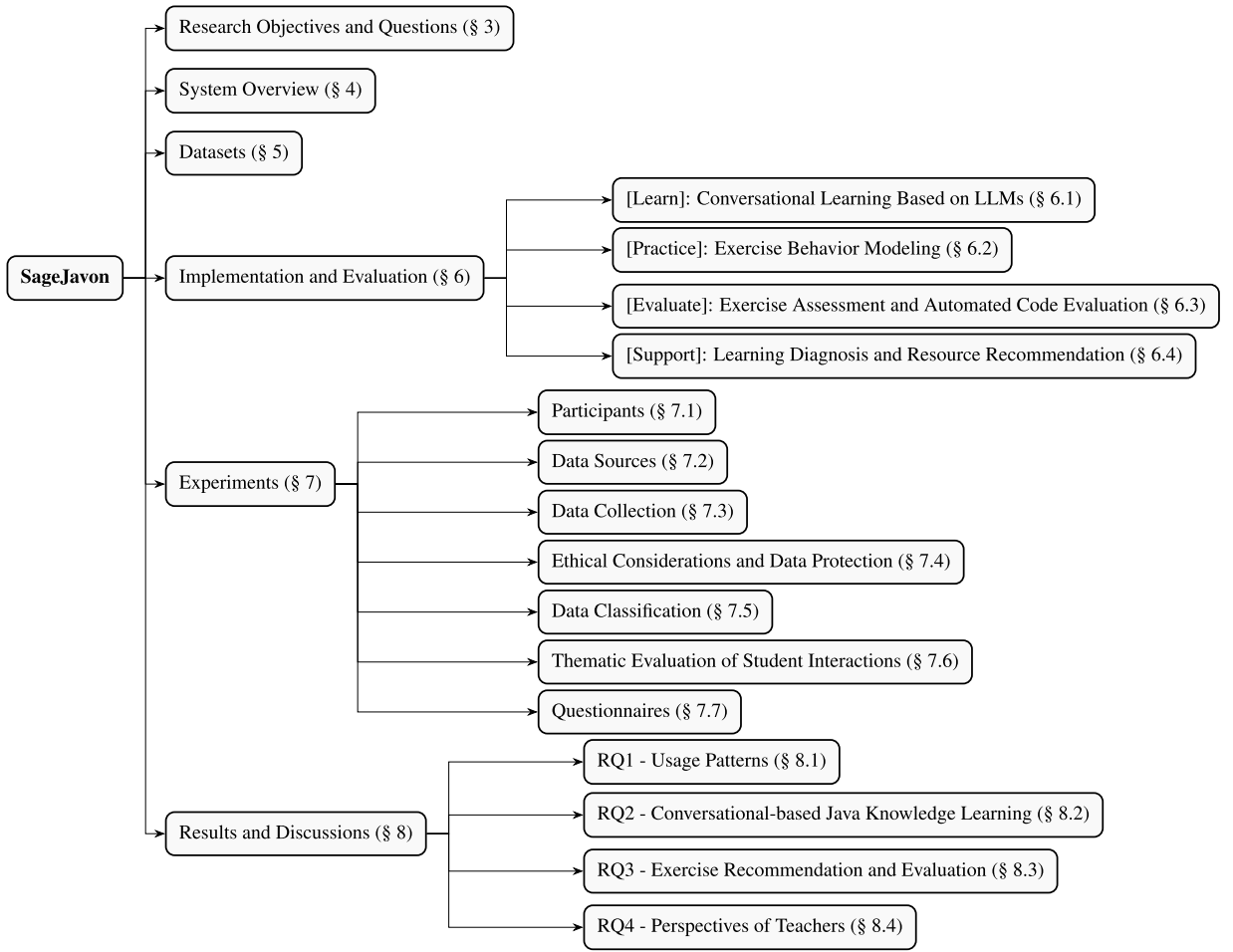


Fig. 1. Mind map of this paper.

2. Integration of Retrieval-Augmented Generation for Course-Specific Adaptability and Heuristic Step-by-Step Thinking and Follow-up Questioning for Computational Thinking. SageJavon leverages LLMs and an improved Retrieval-Augmented Generation (RAG) framework to deliver tailored, multi-modal resources. Instructors can upload custom materials, such as slides and e-books, ensuring a personalized tutoring experience. Additionally, heuristic and follow-up questioning with LLMs guide students step-by-step in problem-solving, fostering computational thinking.

3. Graph-Based Knowledge Tracing for Student Assessment and Automated Code Evaluation. We use graph-based knowledge tracing to assess students' knowledge and predict their learning progress, providing insights into strengths and weaknesses. This integrates with a recommender system to suggest personalized exercises. We also introduce FUPS-Score, an automated evaluation framework for assessing student code and offering personalized feedback.

4. Real-World Classroom Deployment and its Educational Significance in Programming Education. SageJavon outperforms traditional tools, demonstrating lower cognitive load, improved knowledge transfer, and higher engagement. Deployed in real classrooms, it showcases the potential for scalable, plug-and-play educational tools adaptable to various programming languages.

The paper is structured as follows. [Section 2](#) reviews related work, [Section 3](#) outlines the research objectives and questions, and [Section 4](#) presents the architecture and overview of the SageJavon system. [Section 5](#) describes the datasets, [Section 6](#) details the four modules of SageJavon, [Section 7](#) discusses classroom deployments and analysis, [Section 8](#) presents the results, [Section 9](#) compares SageJavon with existing AI tutors, and [Section 10](#) explores the implications and concludes the paper. A mind map is provided in [Fig. 1](#).

2. Related work

2.1. Large pre-trained language models

Large Pre-Trained Language Models (LLMs) have played a key role in chat-based programming education. Most modern LLMs are based on Encoder-Decoder (Vaswani et al., 2017) or Decoder-only (Naik et al., 2024) architectures. The encoder transforms user queries into continuous vector representations, capturing semantic information, while the decoder predicts the corresponding output sequences based on these representations.

Code large language models. Code LLMs have demonstrated exceptional performance in code-related tasks such as code generation (Chen et al., 2022; Hu et al., 2025; Huang et al., 2023), code summarization (Lu et al., 2021), bug repair (Xuan et al., 2016) and code question answering (Li et al., 2024). These models were designed to assist with various code-related tasks.

Encoder-decoder architectures, such as BART (Lewis, 2019) and T5 (Raffel et al., 2020), formed the foundation of many Code LLMs. Building on T5, CodeT5p (Wang et al., 2023), developed by Salesforce, was pre-trained on a diverse set of tasks using both pure code data and code-text paired data. It was capable of handling a wide range of code-related tasks, including code generation, code completion, mathematical programming, and text-to-code retrieval. Additionally, WizardCoder (Luo et al., 2023) enhanced code generation capabilities through Evol-Instruct fine-tuning.

Furthermore, models such as StarCoder (Li et al., 2023), CodeLlama (Roziere et al., 2023), and Deepseek-Coder (Guo et al., 2024a) demonstrated strong performance in various code-related tasks. Beyond statement-level tasks (Zhu et al., 2024), recent advancements in Code LLMs expanded their focus to more complex levels of code generation. For instance, ClassEval (Du et al., 2023) provided a benchmark specifically designed to evaluate the class-level code generation capabilities of Code LLMs. Moreover, research also began exploring project-level code generation and comprehension (Lin et al., 2024), pushing the boundaries of what these models could achieve in large-scale software development.

General-purpose LLMs for code tasks. In addition to specialized code LLMs, general-purpose models like GLM-4 (GLM et al., 2024) and GPT-4 (Achiam et al., 2023) perform well in code tasks. They handled various programming challenges through instruction fine-tuning (Zhang et al., 2023) and In-Context Learning (Dong et al., 2022). Pre-trained on diverse datasets, including code, these models excelled in coding tasks by leveraging vast, cross-domain knowledge, which made them highly adaptable to different languages, frameworks, and problem-solving approaches. Moreover, API access simplified deployment and made the models accessible to developers and educators.

Analysis and limitations. Despite the strong performance of these LLMs in code-related tasks, their application in programming education can be heavily influenced by domain knowledge, especially when teaching specific course content. These models may generate hallucinations or incorrect outputs, even with fine-tuning (Ding et al., 2023). While fine-tuning addressed this issue to some extent, it often led to overfitting. A more effective approach to mitigate such issues involved using Retrieval-Augmented Generation (RAG) (Lewis et al., 2020; Zhao et al., 2024), which allowed the model to access external knowledge bases.

2.2. Intelligent tutoring systems for programming education

Intelligent Tutoring Systems (ITS) (Kazemitabaar et al., 2024) employs an approach that encourages self-directed learning by guiding students toward solutions without directly providing answers. Similarly, CREF (Yang et al., 2024) presents an LLM-based conversational software repair framework designed for programming tutors, tackling the challenge of data leakage while enriching interactions in conversation-driven program repair. In addition, Conversing with Copilot (Denny et al., 2023) explores the pedagogical benefits of using Copilot to assist students in programming tasks, emphasizing its value in fostering deeper learning through interactive problem-solving. Meanwhile, KIAAA (Eilermann et al., 2023) introduces an AI-driven ITS system specifically designed for programming education in automation, capable of both generating and assessing programming tasks. Many AI-based intelligent tutoring systems (ITS) have been used in education, including those discussed in (Liebenow et al., 2025; Liu et al., 2024).

Analysis and Limitations. Despite these, most ITS primarily function as conversational agents that provide guidance through dialogue, rather than offering a comprehensive “Learn-Practice-Evaluate-Support” learning framework. Additionally, their recommendations often lack alignment with specific course curricula, making them less effective in structured educational settings. Furthermore, these systems generally fall short in delivering personalized assessments and tailored recommendations based on individual student progress, limiting their ability to fully adapt to diverse learning needs.

3. Research objectives and questions

We aim to develop SageJavon, an AI-powered tutoring system designed to address gaps in higher education programming instruction. These gaps include the lack of personalized content recommendations and timely, targeted feedback. The overarching goal is to create an AI tutor that reduces cognitive load, facilitates knowledge construction, and offers tailored, course-specific guidance to students. The following research questions guide our study. In Section 8, we conduct data analysis for each of the research questions and provide corresponding answers.

1. RQ1: Student Usage Patterns. What patterns emerged in student usage of SageJavon regarding frequency, choice of features, usage patterns, and the nature of questions posed?

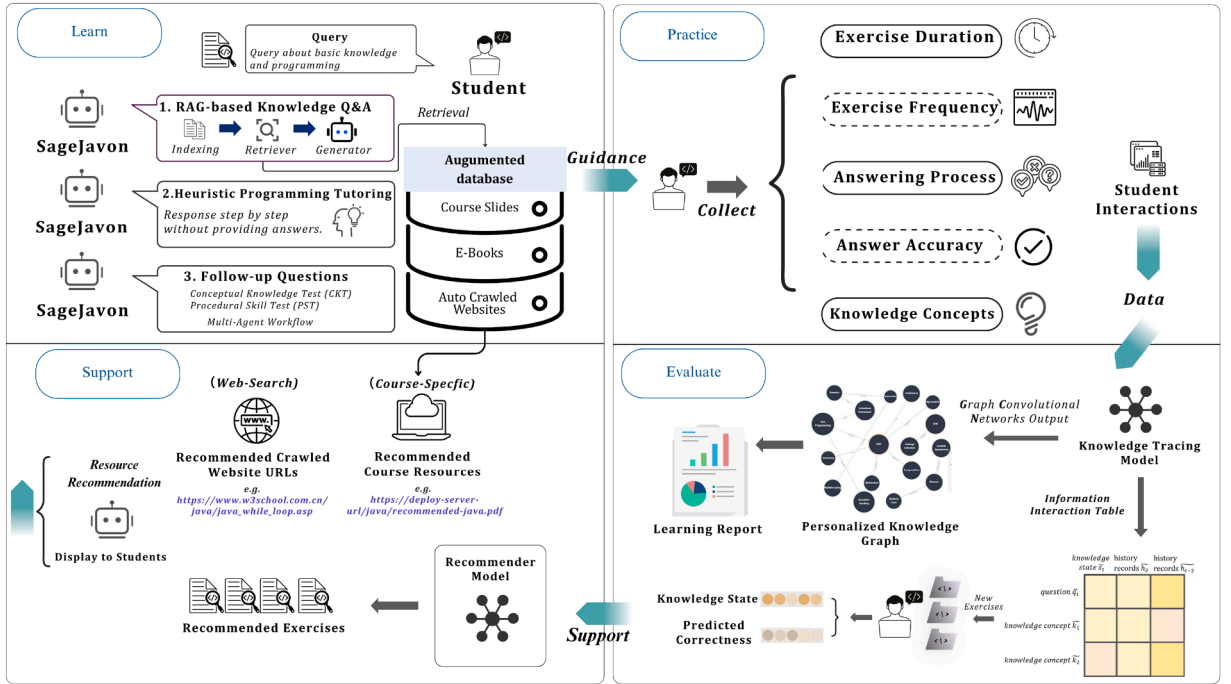


Fig. 2. The architecture of the SageJavon Platform, comprising four main modules: [Learn], [Practice], [Evaluate], and [Support]. (i) Upon entering SageJavon, students can first use the [Learn] module to access the Heuristic Dialogue Learning Module for explanations of Java-related concepts. (ii) If students identify gaps in their knowledge, they can then use the [Practice] module, which recommends targeted exercises for practice. (iii) The [Evaluate] module assesses the student's code based on their exercise history and predicts their knowledge level. (iv) After determining the student's knowledge level, the [Support] module provides a comprehensive learning diagnosis and recommends tailored resources to facilitate further improvement.

2. RQ2: *Conversational-based Knowledge Learning.* To what extent can conversational-based Java knowledge learning facilitate the active construction of knowledge by students and reduce cognitive load? How effective was SageJavon in delivering accurate, course-specific Java knowledge answers and offering algorithmic problem guidance without directly providing solutions?

3. RQ3: *Exercise Evaluation and Recommendation.* Can exercise evaluation and recommendation based on LLMs accurately match the student's learning level and provide appropriate suggestions? How effective was SageJavon in evaluating students' code and recommending exercises that align with their learning progress? Did the recommendations help students improve their performance?

4. RQ4: *Perspectives of Teachers.* What are the perspectives of educators regarding learner-focused AI tutors like SageJavon regarding its integration into the curriculum, recommendations for improvement, and effective pedagogy?

4. System overview

The SageJavon system follows “Learn-Practice-Evaluate-Support” framework as shown in Fig. 2, where each module feeds into the next, creating a dynamic and adaptive learning loop that evolves based on student interactions.

4.1. [Learn] module: Conversational learning

SageJavon begins by guiding students to grasp key Java concepts and computational thinking when programming. When a student submits a query $q(x)$, the system encodes it as a dense vector and uses RAG to match it with relevant content $Z_{\text{retrieved}}$ from the knowledge base. $q(x)$ and $Z_{\text{retrieved}}$ are then processed by the generator to produce answers $A(q(x), Z_{\text{retrieved}})$, as shown in Eq. (1),

$$A(q(x), Z_{\text{retrieved}}) = \text{Generator}(q(x), R(q(x), Z)), \quad (1)$$

where $R(q(x), Z)$ denotes the retrieval function that matches the query $q(x)$ with the relevant content from the knowledge base Z . Additionally, we use our heuristic mechanism to guide students step by step rather than providing direct answers. Besides, we employ follow-up questioning using LLMs to foster deeper engagement.

4.2. [Practice] module: Exercise behavior modeling

As the student engages in exercises, the system tracks their interactions to model their learning behaviors. Key metrics including exercise duration T_i , exercise frequency F_i , answering process $P_i(t)$, answer correctness A_i , and knowledge concepts K_i are collected to create an interaction vector $\mathbf{I}(s_i) = [T_i, F_i, P_i(t), A_i, K_i]$, which monitors the student's progress through the exercises. For each student s_i , the system tracks their unique interaction data $\mathbf{I}(s_i)$, where the components correspond to their individual metrics at time t . The interaction data $\mathbf{I}(s_i)$ is then passed into the subsequent [Evaluate] module for further processing. These data are crucial for tracking ongoing practice and are formally represented in Eq. (2),

$$\mathbf{I}(s_i) = [T_i, F_i, P_i(t), A_i, K_i]. \quad (2)$$

The information gathered in the practice phase informs the evaluation module, bridging the gap between learning and assessment.

4.3. [Evaluate] module: Exercise assessment and automated code evaluation

In the [Evaluate] module, the interaction data $\mathbf{I}(s_i)$ of student s_i , collected from the [Practice] module, is processed by our Graph-based Interaction Model $G(\mathbf{I}(s_i))$. This model predicts the student's knowledge state $\mathbf{k}(s_i)$ and correctness probability $\mathbf{c}(s_i)$, as shown in Eq. (3),

$$\text{GIKT}(\mathbf{I}(s_i)) \Rightarrow (\mathbf{k}(s_i), \mathbf{c}(s_i)). \quad (3)$$

The output knowledge state and correctness probability are crucial for understanding the student's learning progress and are passed to the subsequent [Support] module for generating personalized learning resources.

4.4. [Support] module: Comprehensive learning diagnosis and resource recommendation

To deliver targeted exercise recommendations, our system actively analyzes each student's learning status based on two key metrics produced by the [Evaluate] module: the knowledge state vector $\mathbf{k}(s_j)$ and the correctness probability vector $\mathbf{c}(s_j)$. These vectors represent the estimated mastery of concepts and the likelihood of answering related questions correctly for each student s_j .

The recommendation engine identifies a set of peer students who share similar knowledge profiles with the target student s_i . It then aggregates the knowledge states and performance patterns of these peers to recommend a personalized exercise set E_i that best supports s_i 's learning needs. We define the overall recommendation process as shown in Eq. (4),

$$R\left(\{\mathbf{k}(s_j), \mathbf{c}(s_j)\}_{j \in \text{similar}(s_i)}\right) \Rightarrow E_i. \quad (4)$$

The symbols in Eq. (4) are explained as follows.

- s_i : the target student for whom personalized exercises are recommended.
- s_j : a peer student who exhibits behavioral or conceptual similarity to s_i .
- $\mathbf{k}(s_j)$: the knowledge state vector of s_j , indicating concept-level mastery.
- $\mathbf{c}(s_j)$: the correctness probability vector of s_j , reflecting predicted performance.
- $\text{similar}(s_i)$: the set of peer students selected based on similarity metrics (e.g., cosine similarity).
- $R(\cdot)$: the recommendation function that aggregates peer knowledge to generate suitable exercises.
- E_i : the resulting set of recommended exercises tailored to s_i 's learning needs.

We actively leverages peer patterns to enhance recommendation quality, following a collaborative filtering approach grounded in pedagogical diagnostics described in Section 6.4. The loop depicted in Fig. 3 establishes a continuous cycle in which each phase, including learning, practicing, evaluating, and supporting, drives the next and forms a personalized and adaptive learning experience. Section 6 provides the detailed implementation of the specifics and experiments.

5. Datasets

We describe the dataset for training and evaluating the GIKT model in Section 6.3.1, the dataset for our code evaluation framework in Section 6.3.4, and the data collected post-deployment of the AI tutor in Section 7. A detailed analysis of the deployment data is provided in Section 8.

6. Implementation and evaluation

6.1. [Learn] Module: Conversational learning based on LLMs

We use GLM-4-Plus (GLM et al., 2024) as the core generative module for our AI tutor. By integrating RAG, we enhance response generation with access to external knowledge, enabling more comprehensive and context-aware answers. Our prompt engineering guides ChatGLM to help students solve programming problems step by step, fostering logical reasoning.

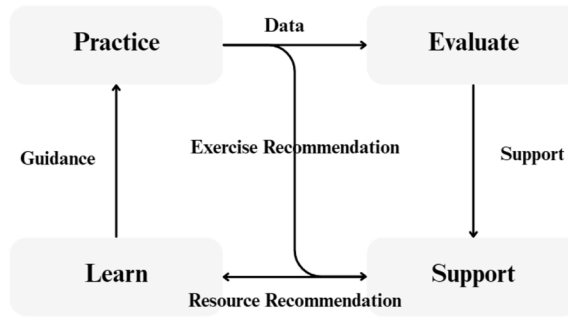


Fig. 3. Illustration of the learning interaction cycle in Sage.Javan. Students engage in practice activities, followed by evaluation of their performance. Based on the evaluation results, the system offers personalized support through targeted resource and exercise recommendations. This support informs the subsequent learning phase, enabling a continuous and adaptive learning process.

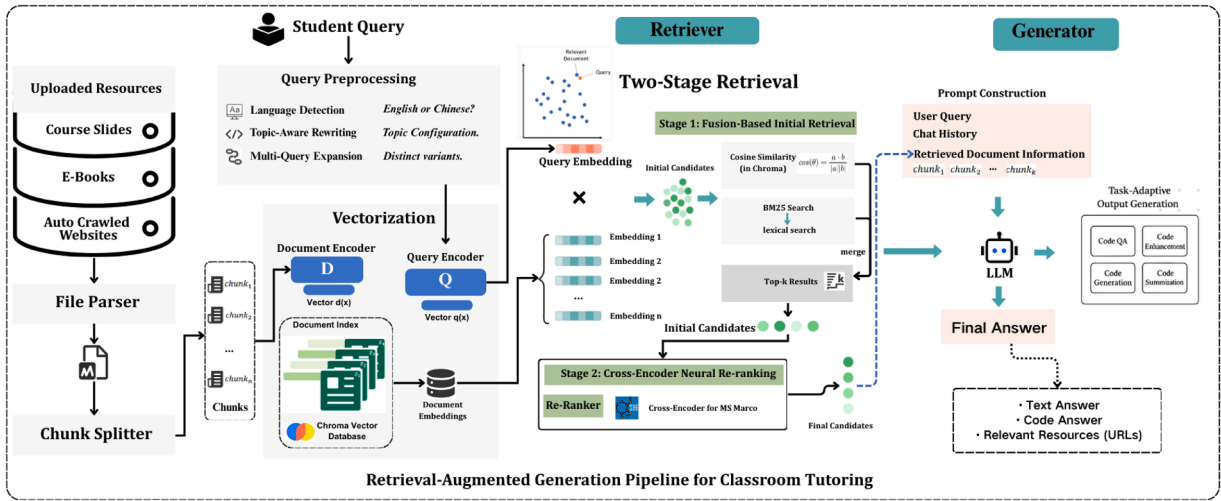


Fig. 4. Architecture of the retrieval-augmented generation pipeline for classroom tutoring. The system integrates multi-source materials and adopts a two-stage retrieval approach combining vector and lexical search, followed by cross-encoder re-ranking and context-aware response generation. Outputs are task-adaptive, supporting text, code, and educational recommendations.

The system supports various output types, including *Code QA*, *Bug Fixing*, *Code Enhancement*, *Code Generation*, and *Code Summarization*, adapting its strategy based on user queries. For example, when asked about a `NullPointerException` in Java, the system provides error identification, a corrected code snippet, and suggestions for avoiding similar issues. Appendix B provides example screenshots of these outputs.

6.1.1. Knowledge Q&A with LLMs powered by retrieval-augmented generation

We implement an augmented RAG architecture tailored to our classroom tutoring scenario, comprising four key stages including *query preprocessing*, *query vectorization*, *two-stage retrieval*, and *response generation*, as illustrated in Fig. 4. Our implementation is based on the open-source project RAG-GPT,² upon which we have conducted substantial modifications and optimizations across all stages to improve retrieval robustness, semantic alignment, and generation fidelity.

1. Query preprocessing. We design a three-stage query preprocessing pipeline: language detection, topic-aware rewriting, and multi-query expansion. (1) *Language Detection*. Using `py3langid`, we detect the query's language (e.g., English, Chinese). This step ensures that subsequent modifications respect language-specific norms and improves template-based changes. (2) *Topic-Aware Query Rewriting*. If the configured topic is not mentioned in the query, we automatically add a language-specific phrase (e.g., “, about 'topic'” in English). This ensures queries remain relevant to the bot's domain, enhancing focus for downstream retrieval. (3) *Multi-Query Expansion*. The original pipeline used a single query per retrieval round, limiting recall and robustness to paraphrased questions. We introduce multi-query expansion by generating three semantically equivalent variants using T5-small model from Huggingface.³

² <https://github.com/gpt-open/rag-gpt>

³ <https://huggingface.co/google-t5/t5-small>

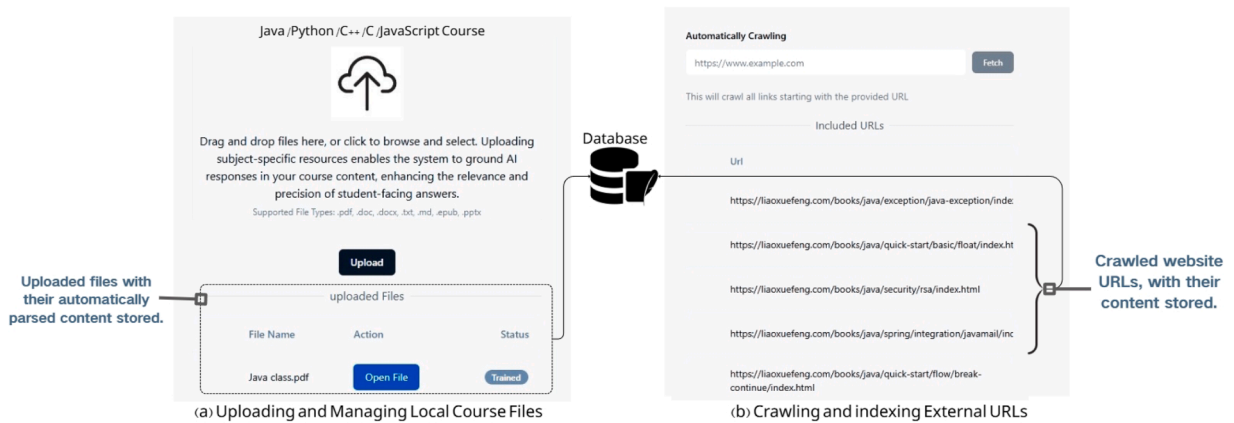


Fig. 5. User Interface of the RAG backend, where teachers can drag and upload course-specific resources or paste learning website URLs. The backend will automatically parse the files or crawl the URLs, and the parsed content will be stored to serve as LLM-augmented information.

We prepend the prompt ‘‘paraphrase:’’ and perform beam search decoding (beam size = 5). This expansion increases document coverage and boosts retrieval precision and recall, while maintaining computational efficiency.

2. Query and document vectorization. We use ZhipuAI’s embedding API⁴ to convert each rewritten query into a dense semantic vector, supporting multiple languages and providing high-dimensional representations for semantic search. For documents, we parse the uploaded files into clean Markdown, segment them into coherent chunks using a language-aware splitter, and embed each chunk individually. These embeddings are stored in the Chroma vector database.⁵ This chunk-based encoding improves retrieval precision by reducing noise and aligning well with short user queries. By combining query expansion with chunk-level document vectorization, our system ensures that semantically related queries and content are closely mapped in vector space, enabling accurate, context-aware retrieval in downstream RAG workflows.

3. Two-stage retrieval: Fusion-based search and cross-encoder re-ranking. To improve both recall and relevance, we transform the original single-vector retrieval into a hybrid pipeline, incorporating semantic fusion and neural re-ranking.

(1) Stage 1: Fusion-based initial retrieval. We merge semantic and lexical signals for hybrid retrieval. First, we perform a cosine similarity search on precomputed query and chunk embeddings in the Chroma database to retrieve semantically relevant results. Simultaneously, we use BM25Okapi⁶ on a dynamically built corpus to find documents with key terms. We then fuse the top-*k* results from both methods, initially assigning equal weight to all documents, with options for more advanced ranking strategies like Reciprocal Rank Fusion (RRF). This fusion combines BM25’s lexical precision with the semantic power of vector search, creating a richer, more relevant candidate set for re-ranking.

(2) Stage 2: Cross-encoder neural re-ranking. We refine the candidate set using the Cross-Encoder model,⁷ which encodes query-document pairs together. This method captures semantic interactions more accurately than independent encodings. Unlike bi-encoders, the Cross-Encoder directly generates a relevance score for each pair, ensuring that the most relevant content is prioritized in the final output. Re-ranking is applied after fusion retrieval to optimize computational efficiency.

In summary, our pipeline introduces three key innovations: (1) hybrid retrieval combining embeddings and BM25 for better recall, (2) Cross-Encoder re-ranking for improved relevance precision, and (3) modular optimizations to ensure high accuracy and low latency in real-time applications.

4. Context-aware and multi-purpose response generation. Our system adapts its response generation strategy to handle different output types based on the user’s query. It supports *Code Q&A*, *Bug Fixing*, *Code Enhancement*, *Code Generation*, and *Code Summarization*. Example outputs are provided in [Appendix B](#).

6.1.2. Flexible backend for course integration

Our RAG backend allows teachers to expand the system’s knowledge base by easily uploading educational materials such as slides (PDF, PowerPoint), e-books (PDF, EPUB), structured notes (Markdown), or external URLs. The system automatically crawls, extracts, and cleans web content. [Fig. 5](#) shows these functions: (a) file upload interface and (b) URL crawling. This plug-and-play design allows easy integration of domain-specific materials, enabling SageJavon to support various subjects beyond Java programming. Teachers can instantly enhance the system’s retrieval capabilities without manual preprocessing, ensuring more accurate and contextually relevant responses.

⁴ <https://www.zhipuai.cn>

⁵ <https://www.trychroma.com>

⁶ <https://pypi.org/project/rank-bm25/>

⁷ <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>

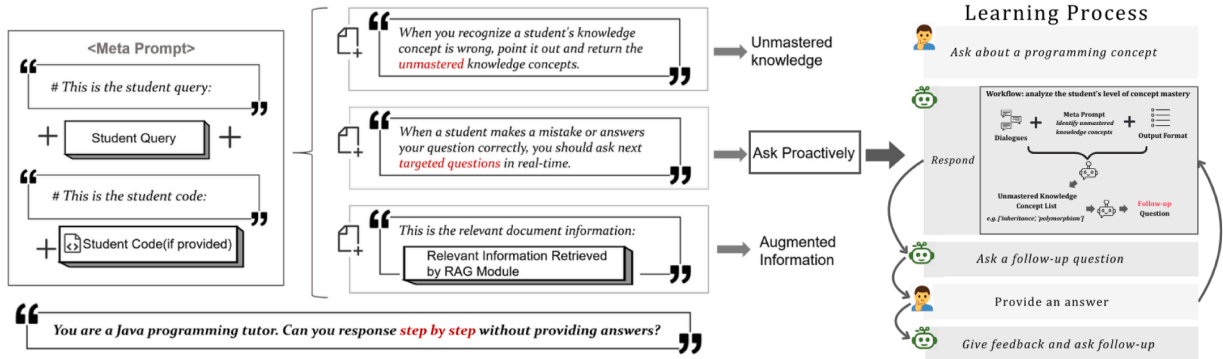


Fig. 6. The prompt structure includes (i) a **Meta Prompt**, which presents the student's query and code, and (ii) an **Augmented Prompt**, which identifies unmastered concepts, proactively asks targeted follow-up questions, and integrates retrieved information to enhance tutoring. The system analyzes the student's concept mastery and actively generates follow-up questions based on this analysis.

6.1.3. RAG evaluation and performance

We assess the performance of our enhanced RAG framework using *Ragas*,⁸ a comprehensive evaluation toolkit for retrieval-augmented generation systems. We evaluate both GLM-series and GPT-series models to quantify the improvements of our enhanced RAG over the baseline approach. Section 8.2.3 provides the experimental details and results, while Appendix F illustrates a sample question evaluation process and its corresponding results.

6.1.4. Programming tutor with heuristic and follow-up questioning

Effective programming education emphasizes both coding and computational thinking (Lu & Fletcher, 2009). We propose a heuristic approach that transitions from observable behaviors (e.g., coding attempts) to deeper cognitive processes. Additionally, the LLM utilizes follow-up questioning (Guo et al., 2024b) to motivate students.

1. Heuristic and proactive questioning strategy. Fig. 6 shows that the prompt structure consists of two main parts. (i) **The meta prompt** provides the AI tutor with the student's query and any supplementary code. It enables the LLM to understand the specific problem the student is encountering. (ii) **Two conditions and an augmented component.** The first condition lets the tutor identify the unmastered knowledge concepts based on the student's response and context. The second condition requires the LLM to proactively ask follow-up questions when necessary. This enables the tutor to guide the student through the learning process in real-time by asking follow-up questions that address any knowledge gaps. (iii) **Augmented information.** The augmented component of the prompt consists of relevant information retrieved by the RAG module in Section 6.1.1, which provides additional document information chunks related to the query.

2. Workflow for generating follow-up questions. As shown in Fig. 6, we design a structured workflow to generate follow-up questions based on the student's concept mastery. The system first takes the student's dialogue history as input, combines it with a meta prompt that explicitly instructs the model to identify unmastered knowledge concepts, and formats the output as a knowledge concept list. Once identifying unmastered knowledge concepts, the AI tutor proactively generates targeted follow-up questions. This process enables the model to adapt to the student's current understanding and refine its tutoring strategy in real-time. By iterating between analysis and questioning, the tutor promotes active engagement and deeper learning.

6.2. [Practice] module: Exercise behavior modeling

We offer 625 Java exercises, consisting of 432 multiple-choice questions and 193 programming exercises, distributed across 12 chapters. Students can choose which exercises to complete based on their preferences or follow recommendations provided by the system according to their individual learning needs. The system collects and analyzes student answer records to better understand their learning behaviors and monitor their progress. These records include essential metrics such as exercise duration T_i , exercise frequency F_i , answering process $P_i(t)$, answer correctness A_i , and the associated knowledge concepts K_i as show in Table 1. These metrics are essential for tracking student s_i 's engagement with the exercises and gaining insights into their learning process.

For each student s_i , the system creates a unique interaction vector $\mathbf{I}(s_i)$, combining the individual metrics to capture their performance and learning trajectory. Each component of this vector represents a specific metric at time t , offering a detailed snapshot of the student's ongoing engagement with the exercises. The system then passes this interaction data to the [Evaluate] module, where the **Knowledge Tracing Module** (Section 6.3) analyzes it to predict the student's knowledge state. This data plays a key role in tracking the student's progress and providing personalized feedback and recommendations.

⁸ <https://github.com/explosiongradients/ragas>

Table 1
Metrics for exercise modeling.

Metric	Description
Exercise Duration T	The total time spent on each practice session.
Exercise Frequency F	The number of practice sessions completed within a specific time frame.
Answering Process $P(t)$	A detailed sequence of cognitive and operational steps taken by the student while attempting to answer a question.
Answer Accuracy A_i	A binary indicator of the correctness of each response, where $A_i = 1$ denotes a correct answer and $A_i = 0$ represents an incorrect answer.
Knowledge Concepts K_j	The specific knowledge concepts or concepts that are assessed by each question.

Table 2
Training data overview: Initial vs Mid-Term.

Data Collection	Total Records	Knowledge Concepts	Students
Initial Training Data	4931	145	548
Mid-Term Training Data	14,526	145	85

6.3. [Evaluate] module: Exercise assessment and automated code evaluation

6.3.1. Knowledge tracing with GIKT model

Using the modeled data from Section 6.2, the Graph-based Interaction Model for Knowledge Tracing (GIKT) (Yang et al., 2020) assesses students' learning progress. GIKT takes $\mathbf{I}(s_i)$ as input, where $\mathbf{I}(s_i)$ represents the interaction data of student s_i , which includes key metrics such as exercise duration, frequency, answering process, correctness, and knowledge concepts. Based on this input, the model typically outputs the student's knowledge state $\mathbf{k}(s_i)$ and predicted correctness $c(s_i)$ for future questions.

1. Model implementation and enhancement. We reproduce GIKT in PyTorch based on the model details provided in the paper and its official repository⁹ and make necessary adaptations to ensure the effective application of the GIKT model in an exercise recommendation system. Few previous studies have successfully integrated Knowledge Tracing (KT) models with recommendation algorithms in real-world production systems.

(1) *Training Data.* The training data for the GIKT model consists of student exercise records collected from the two courses: “Java Core Technologies” and “Advanced Java Core Technologies”. Table 2 shows the training data at the beginning of the deployment and the data at the mid-term stage. We split the dataset we collect into 80% training set and 20% as the test set.

(2) *Training Process.* Different from the original parameters in the paper (Yang et al., 2020), we modify the embedding vector dimensions for questions and knowledge concepts from 100 to 200. We set the learning rate to 0.01, the batch size to 256, the dropout to (0.2, 0.4), and train the model for 1000 epochs. To enhance the Graph-based Interaction Model for Knowledge Tracing (GIKT), we apply 5-fold cross-validation and use cross-entropy as the loss function, as defined in Eq. (5),

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)), \quad (5)$$

where y_i represents the true label (correctness of the student's response), $\hat{y}_i$ is the predicted probability, and N is the total number of samples. After training the model for 1000 epochs, we achieved an accuracy of **79.32%**, surpassing the results reported in the original paper.

2. Model evaluation. We validate the effectiveness of our self-trained GIKT model by conducting comprehensive experiments with our collected training dataset and training methodology. We compare our model against several widely used knowledge tracing models, including Bayesian Knowledge Tracing (BKT) (Corbett & Anderson, 1994), which models a student's knowledge state as a binary variable through Bayesian inference; Deep Knowledge Tracing (DKT) (Piech et al., 2015), the first approach to apply deep learning in knowledge tracing by using Recurrent Neural Networks (RNNs) to model students' knowledge states; and Graph-based Interaction Knowledge Tracing (GIKT) (Yang et al., 2020), which incorporates graph-based dependencies. We evaluate the models on three benchmark datasets: **ASSIST09**,¹⁰ which has 3852 students with 123 knowledge concepts, 17,737 questions and 282,619 exercises, **ASSIST12**,¹¹ which has 2,709,436 exercises with 27,485 students, 265 knowledge concepts and 53,065 questions, and **EdNet**,¹² which has 5000 students with 189 knowledge concepts, 12,161 questions and 676,974 exercises.

Following standard practices in knowledge tracing research, we use accuracy (ACC) as the evaluation metric. The experimental results, shown in Table 3, indicate that our implementation of GIKT achieves accuracy levels comparable to those reported in the original paper, with a slight improvement. This slight increase in accuracy may be attributed to the larger number of training epochs in our implementation. However, when tested in our collected dataset, our implementation of GIKT achieves the highest accuracy among

⁹ <https://github.com/ApexEDM/GIKT>

¹⁰ <https://sites.google.com/site/assistentmentsdata/home/assistentment-2009-2010-data/skill-builder-data-2009-2010>

¹¹ <https://sites.google.com/site/assistentmentsdata/home/2012-13-school-data-with-affect>

¹² <https://github.com/EdNetData/EdNet>

Table 3

Accuracy (ACC) results on four datasets. “original GIKT” refers to Yang et al. (2020), and “Our Dataset” means our collected dataset.

Model	Dataset			
	ASSIST09	ASSIST12	EdNet	Our Dataset
BKT	0.6571	0.6204	0.6027	0.6352
DKT	0.7561	0.7286	0.6822	0.7361
original GIKT	0.7896	0.7754	0.7523	–
our trained GIKT	0.7931	0.7823	0.7612	0.7932

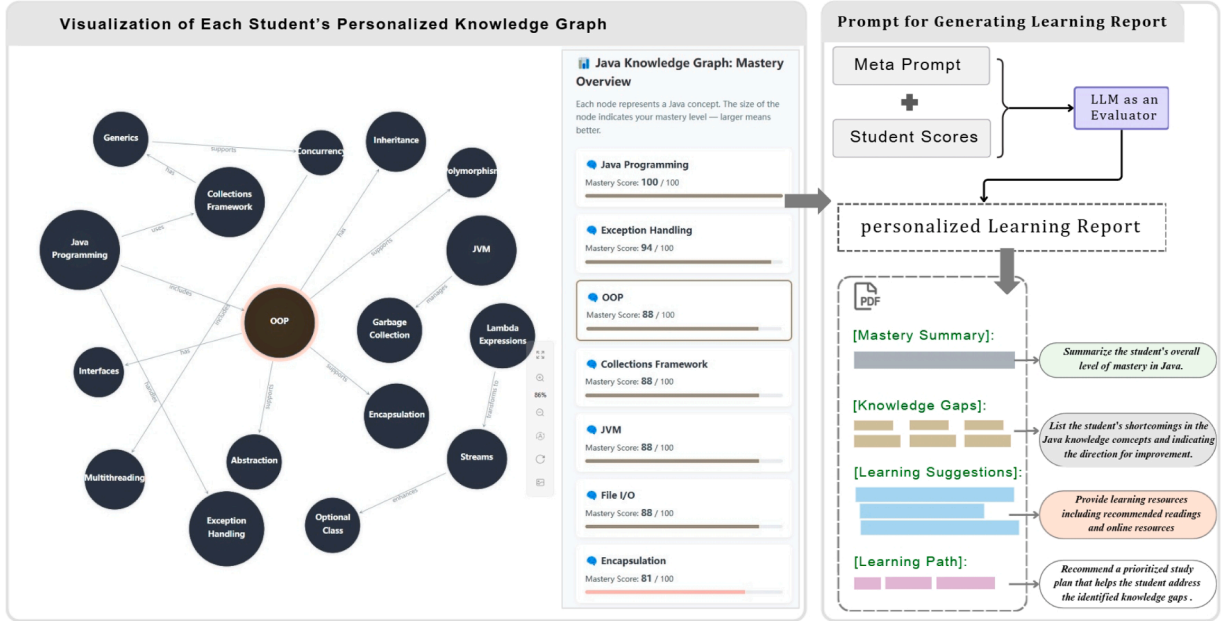


Fig. 7. Visualization of the Java Knowledge Graph, showing the relationships between key Java concepts. Each node represents a concept, with its size indicating the mastery level of the student. Edges represent the dependencies between concepts, such as *includes*, *supports*, and *uses*. The right panel displays the sorted mastery scores for each concept, providing a quantitative overview of the student's proficiency.

all models evaluated. This result suggests that deploying our trained GIKT model into the system for knowledge graph construction and knowledge state prediction can better assess students' knowledge states. Furthermore, this work represents the first attempt to integrate the GIKT knowledge tracing model with recommendation in an exercise recommendation system.

6.3.2. Adapting GIKT for knowledge graph construction and prediction

To adapt GIKT to our system, we modify its output and enhance it with visualization. Instead of predicting probabilities, the model now predicts knowledge state vectors for students. This adjustment allows storing and continuously updating knowledge state vectors, facilitating personalized question recommendations (Section 6.4). As a result, we use GIKT for two primary purposes.

1. Construction of Knowledge Graph. We construct a comprehensive Java Knowledge Graph using adjacency matrices generated by the Graph Convolutional Networks (GCNs) within GIKT. The GCN encodes the question-skill interactions, producing three adjacency structures: (i) *question-question* (*qq-table*), (ii) *skill-skill* (*ss-table*), and (iii) *question-skill* (*qs-table*). These form the graph topology in Fig. 7, where each node represents a Java knowledge concept and edges show semantic dependencies or prerequisites. Node size reflects a student's mastery, with larger nodes indicating better understanding. Edge labels capture pedagogical relationships (e.g., “supports”, “uses”, “extends”) between concepts. Additionally, a sorted summary list of key concepts and mastery scores is provided. This graph serves both as a structural representation of conceptual interrelations and a dynamic diagnostic tool for visualizing learning profiles, helping educators and learners identify knowledge gaps and prioritize study paths. Section 6.3.3 offers a feature for exporting personalized student reports.

2. Prediction. GIKT generates an information interaction table at a specific timestamp t , integrating the student's current knowledge state $\hat{S}_t$, historical knowledge states $\hat{h}_t$ and $\hat{h}_{t-2}$, the target question $\tilde{q}_t$ and relevant knowledge concepts $\tilde{k}_1, \tilde{k}_2$. Here, $\hat{h}_t$ represents the student's knowledge state at time t , which is the embedding of the student's learning state based on prior interactions. The symbol $\hat{h}_{t-2}$ represents the student's knowledge state from two previous time steps, reflecting the historical progression of their learning. The target question $\tilde{q}_t$ represents the new question at time t , while $\tilde{k}_1$ and $\tilde{k}_2$ are the knowledge concepts or skills relevant to the target question. This table captures the relationships between the student's knowledge state, historical exercise performance, and the current

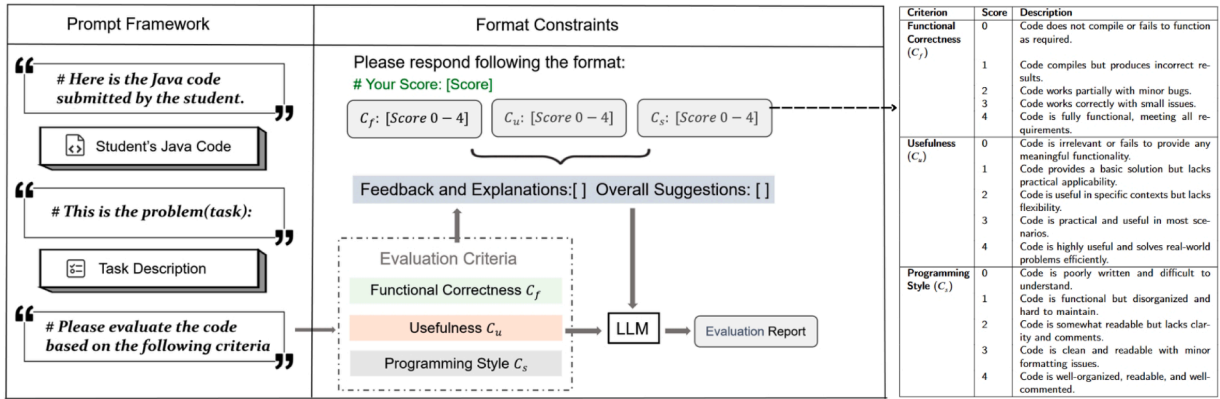


Fig. 8. Illustration of the FUPS-Score automated evaluation pipeline. The framework assesses prompt-generated code by computing its alignment with human-annotated reference scores, integrating multiple correlation-based evaluation metrics.

target question. As shown in the [Evaluate] module of Fig. 2, the table rows represent the knowledge concepts being assessed (e.g., $\tilde{k}_1$ and $\tilde{k}_2$), and the columns represent the target question $\tilde{q}_i$ and the student's historical knowledge states at different time points ($\tilde{h}_i$ and $\tilde{h}_{i-2}$). The interaction between these elements provides a comprehensive view of the student's learning progress, which is used to predict the updated knowledge state vector S_j and the predicted correctness C_j for the student's response to the new exercise. These predictions are then used in recommendation (Section 6.4) to select the next exercise for the student.

6.3.3. Personalized learning report generation using knowledge graphs

Fig. 7 illustrates the workflow for generating personalized learning reports based on each student's knowledge graph. The system encodes concept-level mastery scores into a structured *Meta Prompt*, which instructs the LLM to analyze the student's understanding. The *Input* provides scores for key concepts, while the *Output Format* specifies four sections: *Mastery Summary*, *Knowledge Gaps*, *Study Suggestions*, and *Study Path*. These components collectively guide the model to generate targeted and actionable feedback. Using the prompt and graph-derived input, the model produces a learning report in Markdown format. The report summarizes strengths and weaknesses, offers improvement strategies, and recommends a prioritized study plan.

6.3.4. Automated code evaluation for students using LLMs

We use In-Context Learning combined with LLMs to automate the assessment and feedback process for students' Java programming skills. Inspired by the ICE-Score (Zhuo, 2024), we introduce the **FUPS-Score** evaluation framework to quantitatively assess student code and offer personalized suggestions. The framework consists of three core modules, as shown in Fig. 8: *Prompt Framework*, *Format Constraints*, and *Evaluation Criteria*.

1. Prompt Framework. We design a structured prompt template to standardize input for assessment. This includes (1) the student's submitted Java code, (2) the corresponding task description, and (3) a request for evaluation based on predefined criteria. This structure ensures consistent LLM responses and reliable assessment generation.

2. Format Constraints and Fill in the Blank. We enforce strict format constraints on the model's output to reduce unnecessary token generation. As shown in Fig. 8, the model returns values only within designated placeholders (e.g., "[]") while omitting other fixed elements of the response template. These placeholders are merged with our predefined template to generate the final structured feedback. This approach reduces token consumption while maintaining structured and consistent responses aligned with the evaluation framework.

3. Evaluation Criteria and Scoring Process. Student code is assessed based on three key dimensions: *Functional Correctness* (C_f), *Usefulness* (C_u), and *Programming Style* (C_s). Each criterion is rated on a 5-point scale ranging from 0 (poor) to 4 (excellent). Fig. 8 summarizes the evaluation rubrics for each dimension.

To illustrate the assessment process, we present an example in Table 4. The content enclosed in [] represents the output generated by LLMs. The entire table reflects the final response returned to the student, merging model-generated content with the predefined template. We also provide a series of annotated evaluation examples in Appendix E to demonstrate the application of the FUPS-Score rubric in our classroom deployment. These examples are based on the representative coding task "Rotate Array" and evaluate the three key dimensions-*Functional Correctness* (C_f), *Usefulness* (C_u), and *Programming Style* (C_s)-across the full scoring range from 0 to 4. Each case includes the student's submitted code, the rationale behind the assigned score, and model-generated suggestions for improvement. In Appendix E, Table E.1 offers a comprehensive summary of the FUPS-Score for each code snippet, including scores for each evaluation dimension, detailed explanations, and model-generated suggestions for improvement. Table E.2 provides the full Java code snippets for each student submission referenced in Table E.1. These examples demonstrate the alignment between the LLM's evaluations and the scoring rubrics defined in Fig. 8. They also enhance the transparency, interpretability, and practical reliability of the FUPS-Score framework, making the evaluation process more understandable and actionable.

Table 4

Example of an automated student code evaluation, detailing scores and suggestions for Functional Correctness, Usefulness, and Programming Style, with corresponding code snippets and an overall improvement summary.

Your Score: [92]
Section 1: Functional Correctness (C_f) & Score: [4] Suggestion: <i>[The code meets the basic requirements, but additional interactive elements, such as user input, could improve usability.]</i> <pre>Scanner scanner = new Scanner(System.in); System.out. print ("Please enter the initial value of variable 'a': "); int a = scanner. nextInt ();</pre> As seen in the code example above.
Section 2: Usefulness (C_u) & Score: [3] Suggestion: <i>[The functionality is correct, but adding error-handling mechanisms such as input validation would improve robustness.]</i> <pre>if (! scanner. hasNextInt ()) { System.out. println ("Please enter a valid integer."); return; // or perform other error handling }</pre> As seen in the code example above.
Section 3: Programming Style (C_s) & Score: [4] Suggestion: <i>[To enhance readability and maintainability, consider adding comments explaining the purpose of the code and any breakpoints.]</i>
Overall Suggestion Summary: <i>[The code demonstrates good structure and functionality but could benefit from improved error handling and additional user-friendly features. Consider implementing input validation in all user interactions and adding meaningful comments to enhance readability.]</i>

4. Prompt Performance Evaluation. We evaluate the effectiveness of FUPS-Score using GLM-4-Plus on two datasets: the CoNaLa benchmark for Python and a Java dataset collected from SageJavon. **(1) Python dataset (CoNaLa).** CoNaLa (Yin et al., 2018) is a widely used benchmark for natural language to code generation. It consists of 2,860 annotated Python code snippets. Each snippet has a usefulness score on a five point scale from 0 to 4, with an average of 4.5 human annotators per snippet (Zhuo, 2024). A score of 0 means that the generated code is irrelevant or unhelpful, and a score of 4 means a fully correct and effective solution. These human ratings serve as ground truth and allow us to measure how well automatic evaluation metrics align with human judgments. **(2) Java dataset (SageJavon).** To match our Java oriented deployment scenario, we construct a small instructor annotated Java dataset from SageJavon. We sample several representative Java programming tasks from course assignments and collect a diverse set of anonymized student solutions that cover a wide quality range. In total, the dataset contains about 200 Java submissions. For each submission, two or three teaching assistants independently assign integer scores from 0 to 4 on three dimensions, namely functional correctness (C_f), usefulness (C_u), and style (C_s), according to a shared rubric. We then aggregate these dimension scores into an overall usefulness rating for each submission. We compare our improved evaluation framework with four metrics to systematically analyze the correlation between different evaluation metrics and human ratings

(i) **BLEU** (Papineni et al., 2002) measures n gram overlap between generated and reference code but does not model syntax or semantics.

(ii) **CodeBLEU** (Ren et al., 2020) extends BLEU by integrating syntax aware and data flow based components, which makes it more effective for code evaluation.

(iii) **CodeBERTScore (F1 and F3 variants)** (Zhou et al., 2023) uses pre trained embeddings from CodeBERT to assess similarity between generated and reference code and therefore captures deeper semantic relationships.

(iv) **ICE Score** (Zhuo, 2024) instructs large language models to perform human like multi dimensional code evaluation and achieves strong correlations with subjective human preferences and objective functional correctness.

To quantify the alignment of these metrics and human preferences, we compute three correlation coefficients, Kendall tau, Pearson, and Spearman.¹³ As shown in Table 5, FUPS-Score attains the highest correlation with human usefulness ratings across all three coefficients on both datasets. On the Python CoNaLa benchmark, it improves over ICE Score by 3.2% in Kendall tau, 4.8% in Pearson,

¹³ <https://scipy.org/>

Table 5

Correlation of different metrics with human-preferred usefulness on Python (CoNaLa) and Java (SageJavon) datasets.

Metric	Python (CoNaLa)			Java (SageJavon)		
	Kendall-Tau	Pearson	Spearman	Kendall-Tau	Pearson	Spearman
BLEU	0.423	0.572	0.542	0.395	0.553	0.515
CodeBLEU	0.259	0.397	0.339	0.241	0.382	0.329
CodeBERTScore-F1	0.464	0.579	0.595	0.452	0.571	0.587
CodeBERTScore-F3	0.437	0.549	0.564	0.426	0.541	0.552
ICE Score	0.535	0.627	0.596	0.518	0.613	0.589
FUPS-Score (Ours)	0.567	0.675	0.661	0.549	0.658	0.643

and 6.5% in Spearman. On the Java SageJavon dataset, it still yields gains of about 3.1%, 4.5%, and 5.4% in Kendall tau, Pearson, and Spearman respectively. These consistent improvements over strong baselines suggest that FUPS-Score captures aspects of code quality such as functional correctness, clarity, and syntactic coherence more faithfully than existing automatic metrics, and that this advantage transfers from Python to Java.

6.3.5. Complementing traditional test-case-based evaluation with FUPS-Score.

Traditional test-case-based evaluations focus on verifying a program's basic correctness and functionality by ensuring it meets specific requirements and passes predefined input-output tests. However, they do not evaluate other critical aspects, such as code style, design, or user-friendliness. In contrast, FUPS-Score leverages LLMs to provide a more comprehensive evaluation, offering feedback that goes beyond functional correctness. Rather than replacing traditional unit tests, FUPS-Score complements them by adding a qualitative layer of feedback. While unit tests verify functional correctness, FUPS-Score assesses code from multiple perspectives, including readability and design, giving students a more holistic view of their coding practices.

6.4. [Support] module: Comprehensive learning diagnosis and resource recommendation

We develop a personalized recommendation algorithm that integrates knowledge tracing and user collaborative filtering (KTUCF-PRA), as shown in [Algorithm 1](#). Specifically, we use the GIKT model trained in [Section 6.3](#) to output each student's knowledge state $\mathbf{k}(s_i)$ and correctness probability $\mathbf{c}(s_i)$. When recommending exercises for a student i , the process consists of three steps.

Algorithm 1: Personalized recommendation algorithm based on knowledge tracing.

Input: Student's historical answering record S_i , Number of recommended exercises N , Question-to-question relation matrix $\mathbf{A}_{EE}$, Similarity threshold $\theta = 0.95$, Exercise relation threshold τ .

Output: Recommended exercise set R .

```

1 begin
2    $\mathbf{k}(s_i), \mathbf{c}(s_i) \leftarrow \text{GIKT}(S_i)$  // Compute the knowledge state vector and correctness probability for student  $i$ .
3    $Q \leftarrow S_i$  // Initialize the candidate exercise set with student's historical records.
4    $KS \leftarrow \{\mathbf{k}(s_j) \mid j \in \text{all students}\}$  // Compute knowledge states for all students.
5   for  $\mathbf{k}(s_j) \in KS$  do
6      $\text{sim}(i, j) \leftarrow \frac{\mathbf{k}(s_i) \cdot \mathbf{k}(s_j)}{\|\mathbf{k}(s_i)\| \|\mathbf{k}(s_j)\|}$  // Compute cosine similarity between students  $i$  and  $j$ .
7     if  $\text{sim}(i, j) \geq \theta$  then
8        $Q \leftarrow Q \cup S_j$  // Add answering records of similar students to the candidate set to student  $i$ .
9    $Q_{\text{rel}} \leftarrow \{q' \mid \mathbf{A}_{EE}(q, q') > \tau, q \in Q\}$  // Retrieve related exercises using relation matrix
10   $Q \leftarrow Q \cup Q_{\text{rel}}$  // Augment candidate set with related exercises.
11   $P(\hat{y}_k \mid \mathbf{k}(s_i)) \leftarrow \text{GIKT}(\mathbf{k}(s_i), Q)$  // Predict answering probabilities for candidate exercises.
12   $Q_{\text{sorted}} \leftarrow \text{Sort}(Q, \text{ascending order of } P(\hat{y}_k \mid \mathbf{k}(s_i)))$  // Sort exercises by predicted difficulty.
13   $R \leftarrow \text{mid}_{Q_{\text{sorted}}, |Q|=N}$  // Select middle range exercises for balanced challenge.
14 return  $R$  // Output the recommended exercise set.
```

1. Identifying Similar Users. We determine similar users based on the target student's knowledge state $\mathbf{k}(s_i)$, which is an output of the GIKT model:

$$\text{GIKT}(\mathbf{I}(s_i)) \Rightarrow (\mathbf{k}(s_i), \mathbf{c}(s_i)).$$

The vector $\mathbf{k}(s_i) = [k_1^i, k_2^i, \dots, k_m^i]$ represents the knowledge state of student i , where each element k_r^i corresponds to their mastery level of the r th knowledge concept. To measure the similarity between the target student j and another student i , we compute the cosine similarity as follows:

$$\text{sim}(i, j) = \frac{\mathbf{k}(s_i) \cdot \mathbf{k}(s_j)}{\|\mathbf{k}(s_i)\| \|\mathbf{k}(s_j)\|},$$

where $\mathbf{k}(s_i)$ and $\mathbf{k}(s_j)$ are the knowledge state vectors of students i and j , respectively. The notation $\|\mathbf{k}(s)\|$ represents the Euclidean norm (magnitude) of the vector, defined as:

$$\|\mathbf{k}(s)\| = \sqrt{\sum_{t=1}^m (k_t^s)^2},$$

which measures the overall strength of a student's knowledge state across all m knowledge concepts. We select student i as a similar user to student j if their similarity score satisfies $\text{sim}(i, j) \geq \theta$, where $\theta = 0.95$ is the predefined similarity threshold. We choose a relatively high threshold $\theta = 0.95$ so that the peer set S_i only contains students whose GIKT-based knowledge states are very close to the target student, rather than a large group of loosely related peers. A small sensitivity analysis on a held-out split ([Appendix D](#)) shows that lower thresholds increase the size of S_i but do not consistently improve offline recommendation quality, which supports this choice.

2. Constructing the Candidate Exercise Set. To generate personalized exercise recommendations, we first retrieve the exercise records of the target student i and their similar students identified in the previous step. Let E_i^{his} denote the set of exercises previously attempted by student i , and let E_s^{his} represent the exercise records of similar students $s \in S_i$, where S_i is the set of similar students with similarity above a predefined threshold. The initial candidate exercise set is then formed by merging these records:

$$E_i^{\text{cand}} = E_i^{\text{his}} \cup \bigcup_{s \in S_i} E_s^{\text{his}}.$$

To further refine the candidate set, we incorporate additional exercises based on the question-to-question relation matrix $\mathbf{A}_{EE}$, which captures the relationships between different exercises. For each exercise $q \in E_i^{\text{cand}}$, we identify its related exercises by selecting those with similarity scores exceeding a predefined threshold τ :

$$E_i^{\text{rel}} = \bigcup_{q \in E_i^{\text{cand}}} \{q' \mid \mathbf{A}_{EE}(q, q') > \tau\}.$$

The final candidate exercise set is then obtained by combining the initial candidate set and the related exercises:

$$E_i^{\text{final}} = E_i^{\text{cand}} \cup E_i^{\text{rel}}.$$

3. Ranking and Selecting Personalized Exercises. After constructing the final candidate exercise set E_i^{final} , we proceed with ranking and selection. Specifically, we use the GIKT model to estimate the probability that student i correctly answers each exercise e_k :

$$P(\hat{y}_k | \mathbf{k}(s_i)) = \text{GIKT}(\mathbf{k}(s_i), e_k).$$

The predicted probabilities indicate the student's expected performance on each exercise. To provide an optimal challenge, we sort the exercises in ascending order and select a middle range where the student has a moderate chance of success. This balances reinforcing weaker concepts with achievable progress. The final recommended set is:

$$\hat{E}_i = \text{mid}_{E_i^{\text{final}}, |E|=N} P(\hat{y}_k | \mathbf{k}(s_i)),$$

where **mid** selects the middle range of exercises from the sorted list, ensuring the recommendations challenge the student appropriately. The recommendation process leverages GIKT's structural insights and the student's knowledge state to provide personalized, adaptive learning that reinforces weaker areas and supports learning progression. We interpret this selection rule through Vygotsky's Zone of Proximal Development (ZPD) ([Shabani et al., 2010](#)), which separates tasks a learner can already solve independently from tasks that remain out of reach even with support. For each candidate exercise e_k , the GIKT model predicts $P(\hat{y}_k | \mathbf{k}(s_i))$ for student i and we treat this value as a student-specific difficulty signal. Very high probabilities indicate tasks below the student's ZPD, and very low probabilities indicate tasks beyond the ZPD. The middle probability band is not a global "medium" difficulty label but a set of items that GIKT judges as neither fully mastered nor unreachable for that student. By selecting exercises in this band, the **mid** operator approximates the student-specific ZPD and aligns recommendations with emerging skills that targeted tutoring and scaffolding can strengthen.

7. Experiments

We deploy SageJavon in a *Java Object-Oriented Programming* course for second-year university students in the 2024–2025 first semester. The course covers foundational Java programming concepts including Java environment setup, basic class structures, OOP principles, inheritance, interfaces, and abstract classes. Emphasizing core principles through extensive examples, the course ensures that students gain a solid understanding of Java syntax, the ability to comprehend standard Java programs, and hands-on coding experience.

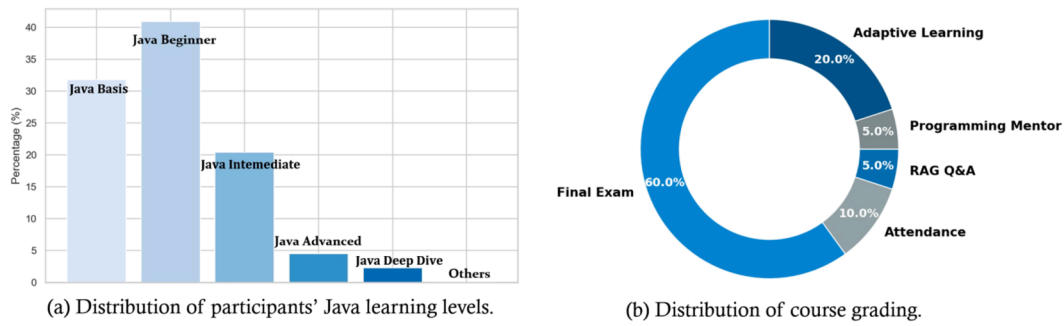


Fig. 9. Distribution of participants' Java learning levels.

7.1. Participants

The course assessment combines traditional grading (60% final exam, 10% attendance) with SageJavon engagement metrics (30%), as shown in Fig. 9(b). Fig. 9(a) visualizes participants' Java proficiency levels, which are predominantly beginner-to-intermediate learners.

7.2. Data sources

To gather a comprehensive understanding of student and educator experiences and interactions with SageJavon, we employ a multi-source data collection strategy including SageJavon's 12 weekly *interaction logs*, *feedback surveys*, an *anonymous post-course survey* (administered after final grade submissions), and *semi-structured interviews* with 10 randomly sampled students as shown in. The detailed information from these datasets is presented in the results section, corresponding to each research question.

7.3. Data collection

We collect four types of data from students within the system: (i) behavioral data, logged by Elasticsearch,¹⁴ including metrics like page clicks, time spent on specific pages, and button interactions, (ii) student queries during conversational learning, paired with system responses, (iii) student responses within the system, and (iv) data from questionnaires and semi-structured interviews (Adeoye-Olatunde & Olenik, 2021) conducted with the students.

7.4. Ethical considerations and data protection

To ensure compliance with ethical guidelines and safeguard privacy, we anonymize all student interaction data, including chat logs, system usage, and exercise submissions. We do not retain any personally identifiable information (PII) and assign pseudonymous identifiers to each participant. The university's Institutional Review Board (IRB) approved the study, and we informed students about the data collection, its purpose, and their rights, including the right to withdraw without consequence. We obtained informed consent from all participants, and the data is securely stored in encrypted repositories with restricted access to ensure privacy protection.

7.5. Data classification

To address RQ1, in the [Learn] module, both the Knowledge Q&A and Programming Tutor modules integrate large language models (LLMs) with manual verification to classify and quantify student inquiries. This process is automated using the ChatGPT-4o API, which categorizes each student conversational interaction inquiry into predefined types based on content analysis.

7.6. Thematic evaluation of student interactions

To address RQ2 and RQ3, we adopt a hybrid evaluation methodology that combines LLMs with manual scoring. This approach is applied to a corpus of 5306 Knowledge Q&A interactions and 5144 Programming Tutor interactions, as summarized in Table 7. The thematic analysis integrates LLM-generated insights with manual verification, evaluating both student queries and the tutor's corresponding responses.

¹⁴ <https://www.elastic.co>

Table 6

Prompt template for scoring dialogue interactions across multiple dimensions. The template integrates the student(*question*)-tutor(*answer*) conversation with predefined evaluation criteria.

Dimension	Label (Component)
Meta Prompt:	Score the answer based on the following criteria:
Correctness (<i>C</i>)	Evaluates the accuracy of the answer.
Helpfulness (<i>H</i>)	Assesses the extent to which the answer is supportive and informative.
Truthfulness (<i>T</i>)	Examines the relevance of the answer to real-world knowledge.
Effectiveness (<i>E</i>)	Measures how well the answer addresses the asker's needs.
Avoidance of Direct Code Solutions (Heuristic) (<i>A</i>)	Assesses how well responses avoid providing direct code solutions, encouraging critical thinking by offering explanations, guidance, or pseudo-code.
Example:	Asker: What is inheritance in Java? Responder: Inheritance in Java allows a class to inherit properties and methods from another class, enabling code reuse. Score: [85, 80, 85, 90, 92]
Next Interaction:	#{ The next student question and tutor answer interaction.}

7.6.1. Score dimensions

Each interaction is scored across several thematic dimensions, as detailed in Table 6. The scoring process follows the template in Table 6. Interactions are manually reviewed, and if the score exceeds the predetermined “Pass Score”, they are deemed valid. Table C.1 provides an example of the scoring process. The evaluation results are then compared to the threshold to determine whether the interaction meets the required quality standards. Valid interactions proceed to the semantic analysis phase.

7.6.2. Score process

The dialogue evaluation executes through four sequential phases: (1) *Template Population*: embedding student questions (*question*) and tutor answers (*answer*) into the scoring template (Table 6) using `{}` placeholder replacement; (2) *LLM Analysis*: automated scoring by ChatGPT-4o, assessing multiple dimensions of quality (e.g., correctness *C*, helpfulness *H*); (3) *Score Export*: generating numerical scores across all metrics; and (4) *Quality Control*: manual verification against the 80-point passing threshold. This integrated framework ensures consistency in evaluation while maintaining academic rigor through human-AI verification.

7.7. Questionnaires

To address RQ2-RQ4, we conducted a comparative experiment with 85 students, evaluating SageJavon against a traditional search tool (Baidu) across 10 carefully selected Java concepts, ranging from fundamental to intermediate levels. The experiment followed a *crossover design*, where participants alternately used both tools to understand Java concepts and complete several tasks. We measured key metrics such as completion time, help-seeking frequency, and task success. After completing the tasks, participants filled out post-task questionnaires to assess cognitive load (using NASA-TLX) and knowledge construction (using a Likert scale). In the final survey, participants compared their experiences with the two tools. This design enables a direct comparison of learning efficiency and cognitive impacts between conversational and search-based programming assistance.

8. Results and discussions

8.1. Results on RQ1 - usage patterns

We analyze how students use SageJavon, including frequency, features, methods, and the types of questions they ask, to understand AI tool usage in different learning contexts.

8.1.1. Student interaction with conversational inquiries: Knowledge Q&A and programming tutor (Section 6.1.1).

A total of 85 students interacted with SageJavon, asking 5520 questions in the Knowledge Q&A module and 5258 questions in the Programming Mentor module. We categorized the questions following the process in Section 7.5 and recorded the frequency for each category, as summarized in Table 7. In the Knowledge Q&A module, the most frequent inquiries pertain to *Java Basics* ($n = 3,142$), while in the Programming Tutor module, the most common questions focus on *Java Programming Practice* ($n = 2,514$). This suggests that students primarily focus on *Java basics* and *programming practice* when asking questions.

Furthermore, we monitor temporal variations in student inquiries. As shown in Fig. 10(a), both the total number of inquiries and the *daily unique student count* exhibit distinct trends, with inquiry spikes occurring around mid-term and final exams. This pattern indicates that students increasingly engage with the system for additional knowledge and programming tutor in preparation for assessments.

Table 7

Categorization of questions in the Knowledge Q&A and Programming Mentor modules. *Type* refers to the task type of the question in each category, including whether the input and output are text or code. *Count* indicates the number of questions in each category. The *Total Count* column represents the sum of both modules.

Module	Category	Type	Count	Total Count
Knowledge Q&A	Java Basics	text-to-text	$n = 3,142$	$n = 5,520$
	Java Programming-Related Questions	text-to-text	$n = 1,494$	
	Tool and Environment Questions	text-to-text	$n = 132$	
	Java Code Debugging and Problem Solving	text-to-code	$n = 538$	
	Irrelevant or Invalid Questions	-	$n = 214$	
Programming Tutor	Java Basics	text-to-text	$n = 1,470$	$n = 5,258$
	Algorithms and Data Structures	text-to-code	$n = 1,105$	
	Java Programming Practice	text&code-to-code	$n = 2,514$	
	Java Code Debugging and Problem Solving	text-to-code	$n = 55$	
	Irrelevant or Invalid Questions	-	$n = 114$	
Grand Total			$n = 10,798$	$n = 10,798$

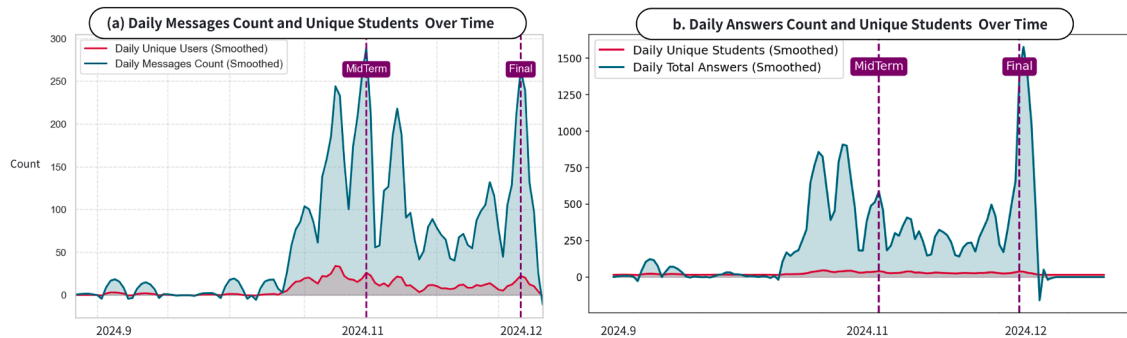


Fig. 10. (a) Daily Messages Count and Unique Students Over Time; (b) Daily Answers Count and Unique Students Over Time. Both plots show smoothed data, with the mid-term and final exam periods highlighted. “Daily Unique Student” refers to students who have interacted at least once during that period.

8.1.2. Exercise and problem-solving engagement: Exercise recommendation and automated assessment of students’ code (Section 6.3~6.4).

We offer 625 exercises in total, including 432 multiple-choice questions and 193 programming exercises. Students completed 26,828 attempts on multiple-choice questions and 2212 attempts on programming exercises, demonstrating substantial interaction with the system. The exercise engagement patterns align with question inquiry patterns, with notable activity peaks around mid-term and final exam periods (Fig. 10(b)). This suggests that students use SageJavon for additional practice to reinforce their knowledge and refine their algorithmic skills, especially during peak academic periods. Smaller engagement peaks throughout the semester may correlate with weekly assignment deadlines, which encourage students to engage with the platform to complete their exercises on time. These usage patterns indicate that SageJavon effectively reinforces Java knowledge and develops practical coding skills through both multiple-choice and programming exercises.

8.1.3. Student perspectives from semi-structured interviews

To complement the log-based analysis, we conducted semi-structured interviews with 10 randomly sampled students using SageJavon throughout the semester. These interviews explored students’ motivations, perceived benefits, challenges, and integration of the system into their study routines. Three recurring themes emerged from the analysis, providing deeper insights into the usage patterns observed in Section 8.1.

1. Exam-driven and assignment-driven usage. Students primarily used SageJavon before major assessments and while working on assignments. This was reflected in the peaks in question inquiries and exercise attempts around assessment periods (Fig. 10). Students mentioned using SageJavon to fill knowledge gaps before exams and clarify concepts from assignments. For example, one student said, “I used SageJavon almost every day before the mid-term to review topics like inheritance and exceptions.”

2. Conversational Q&A for conceptual understanding. Many students preferred the Knowledge Q&A module to understand and review core Java concepts. This preference matched the high frequency of *Java Basics* questions in the logs. Students valued the ability to ask follow-up questions and receive clear, detailed explanations. One student noted, “I could rephrase concepts in different ways to really understand them, which wasn’t always possible in class.”

3. Exercises for knowledge testing and consolidation. Students used the exercise modules to test their understanding after reviewing concepts in the Q&A module. This mirrored the log data showing combined usage of Q&A and exercises. One student said, “If I could solve the exercises with some guidance, I felt more prepared for the exam.”

[RQ1] Insights

(1) *Constructivist Learning Reflected in Query Patterns.* The frequent inquiries related to *Java Basis* ($n = 3,142$) and *Java Programming Practice* ($n = 2,514$) in both the Knowledge Q&A and Programming Tutor modules suggest that students are focused on building foundational and applied Java programming skills. These patterns align with *constructivist learning theory*, which asserts that learning is a dynamic process in which students actively construct their knowledge through the interaction of new experiences and existing cognitive structures. According to this theory, students' learning is most effective when they engage with authentic tasks and collaborative dialogue, as they do through SageJavon. The significant volume of questions indicates that students are actively constructing their understanding of Java by engaging with both theoretical concepts and practical applications, reinforcing their knowledge through iterative inquiry and problem-solving. Section 8.2.1 will present a quantitative analysis of these patterns to further explore student engagement and learning progress.

(2) *Adapting to Students' Learning Progression with Scaffolding Support and Time-Sensitive Usage Patterns.* The increase in inquiries and exercise attempts, especially during mid-term and final periods, shows that SageJavon effectively aligns with students' learning needs by providing scaffolding support. Students use SageJavon not only to clarify doubts but to reinforce key knowledge concepts, underscoring its role in exam preparation. However, the spikes in activity around exam periods indicate that the system is mainly used for last-minute reinforcement, suggesting an opportunity to encourage more consistent engagement throughout the term. By promoting ongoing interaction, SageJavon could help reduce reliance on cramming and enhance long-term retention.

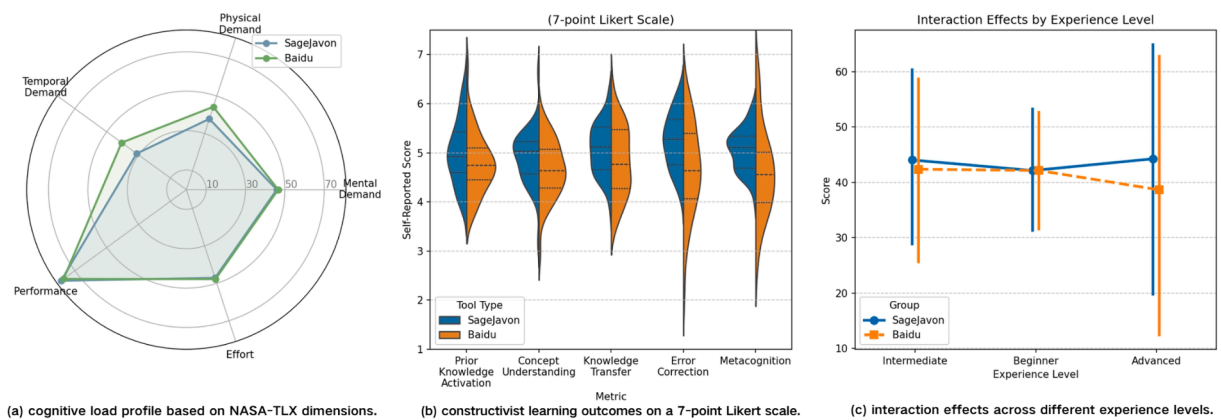


Fig. 11. Questionnaire results visualization.

8.2. Results on RQ2 - conversational-based java knowledge learning

8.2.1. Results of the questionnaire comparing educational theories

In order to answer “To what extent can conversational-based Java knowledge learning facilitate the active construction of knowledge by students and reduce cognitive load?”, we compared the effectiveness of conversational-based Java knowledge learning (SageJavon) with a traditional learning tool (Baidu¹⁵) using a questionnaire that focused on cognitive load, constructivist learning outcomes, and user engagement. The results, shown in Table 8, highlight the comparative effectiveness of both tools. (1) For *Cognitive Load*, SageJavon consistently outperforms Baidu in reducing cognitive load. It shows significantly lower scores in Mental Demand (45.89 vs. 46.64), Physical Demand (37.82 vs. 44.23), and Temporal Demand (31.19 vs. 40.64). SageJavon provides a more user-friendly learning experience, especially for those who may struggle with traditional tools. The significant p -values (e.g., Physical Demand, $p < .001$) highlight the statistical relevance of these differences. (2) For *Constructivist Learning Outcomes*, SageJavon performs better than Baidu in promoting these outcomes. It scores higher in Prior Knowledge Activation (5.01 vs. 4.78), Concept Understanding (4.91 vs. 4.66), and Knowledge Transfer (5.09 vs. 4.82). The differences in Concept Understanding ($p = 0.045$) and Knowledge Transfer ($p = 0.049$) are statistically significant, demonstrating that SageJavon enhances knowledge retention and application. (3) For *Error Correction* and *Metacognition*, SageJavon shows higher scores in both metrics, further reinforcing its advantages. These results indicate that SageJavon supports students in self-reflection and error correction more effectively than Baidu. (4) For *Engagement*, SageJavon is slightly ahead of Baidu in System Usability (81.09 vs. 79.13), though the p -value (0.187) is not significant. The difference in Recommendation Likelihood (4.35 vs. 4.19) is more noticeable, but the p -value (0.258) suggests no significant difference, meaning both tools are viewed similarly in terms of recommendation likelihood.

¹⁵ <https://www.baidu.com> is a traditional search tool widely used for information retrieval.

Table 8

Comparison of Cognitive Load, Constructivist Learning, and Engagement Metrics between SageJavon and Baidu. The values represent the mean score for each metric, followed by the standard deviation (SD) in parentheses, which indicates the variability of the scores within each group.

Cognitive Load						
Metric	SageJavon Mean (SD)	Baidu Mean (SD)	t-statistic	p-value	Cohen's d	Hedges' g
Mental Demand	45.89 (1.54)	46.64 (1.80)	-2.06	0.043	-0.45	-0.44
Physical Demand	37.82 (1.43)	44.23 (1.42)	-20.70	$p < .001$	-4.49	-4.45
Temporal Demand	31.19 (1.51)	40.64 (1.27)	-31.19	$p < .001$	-6.77	-6.72
Performance	78.70 (1.27)	77.12 (1.29)	5.68	$p < .001$	1.23	1.22
Effort	47.10 (1.51)	47.76 (1.29)	-2.20	0.031	-0.48	-0.47
Frustration	43.24 (1.51)	44.29 (1.54)	-3.18	0.002	-0.69	-0.68
Constructivist Learning Outcomes						
Metric	SageJavon Mean (SD)	Baidu Mean (SD)	t-statistic	p-value	Cohen's d	Hedges' g
Prior Knowledge Activation	5.01 (0.65)	4.78 (0.62)	1.68	0.098	0.36	0.36
Concept Understanding	4.91 (0.55)	4.66 (0.58)	2.03	0.045	0.44	0.44
Knowledge Transfer	5.09 (0.51)	4.82 (0.70)	2.00	0.049	0.43	0.43
Error Correction	5.24 (0.56)	4.68 (0.92)	3.44	0.001	0.74	0.73
Metacognition	5.06 (0.42)	4.63 (0.90)	2.83	0.006	0.61	0.60
Engagement Metrics						
Metric	SageJavon Mean (SD)	Baidu Mean (SD)	t-statistic	p-value	Cohen's d	Hedges' g
System Usability	81.09 (6.76)	79.13 (6.84)	1.33	0.187	0.29	0.29
Recommendation Likelihood	4.35 (0.60)	4.19 (0.70)	1.14	0.258	0.25	0.24

Table 9

Comparison of evaluation metrics between RAG-enhanced and heuristic-based tutoring modules. Columns report the average score, valid count (score ≥ 80), standard deviation (Std Dev), and 95% confidence interval (CI). .

Module	Evaluation Dimension	Mean Score	Valid Count (≥ 80)	Std. Dev.	CI Lower	CI Upper
Knowledge Q&A (with RAG)	Correctness	88.87	1447	4.16	88.64	89.10
	Helpfulness	87.87	1431	5.02	87.59	88.14
	Realism	90.34	1296	3.89	90.13	90.56
	Effectiveness	85.25	1372	4.84	84.99	85.51
	Heuristic	84.60	1300	5.10	84.30	84.90
Programming Tutor (Heuristic, no RAG)	Correctness	76.23	483	6.48	75.65	76.80
	Helpfulness	79.89	488	5.76	79.42	80.36
	Realism	82.15	487	5.20	81.72	82.58
	Effectiveness	83.10	493	5.12	82.58	83.62
	Heuristic	84.72	512	4.71	84.31	85.13

Fig. 11 shows that SageJavon outperforms Baidu in reducing cognitive load, especially in Mental Demand, Physical Demand, and Temporal Demand. Fig. A.1 in Appendix A illustrates the connections between cognitive load, learning outcomes, and engagement metrics.

In conclusion, the questionnaire results show that SageJavon provides a more engaging learning experience than Baidu. SageJavon aligns with Cognitive Load Theory and Constructivist Learning Theory by lowering cognitive load and enhancing constructivist learning outcomes. Its strong performance across all metrics indicates it is an effective tool for learning Java, offering a more intuitive experience for active knowledge construction. These findings emphasize the value of conversational-based learning and suggest that SageJavon can improve both engagement and learning outcomes in Java.

8.2.2. Multi-dimensional scoring analysis

In order to answer “How effective was SageJavon in delivering accurate, course-specific Java knowledge answers and offering algorithmic problem guidance without directly providing solutions?”, we use the scoring process described in Section 7.6 with LLMs to evaluate the effectiveness of SageJavon's responses in the Knowledge Q&A and Programming Tutor modules across multiple dimensions including *Correctness* (C), *Helpfulness* (H), *Realism* (R), *Effectiveness* (E), and *Avoidance of Direct Code Solutions* (A). Table 9 shows the scoring results for all evaluated dimensions. Each row shows the **Average Score**, which reflects the overall response quality for each category. A higher score indicates better model performance. The **Valid Count** represents the number of samples with acceptable ratings (scores ≥ 80), providing a measure of high-quality responses. The **Standard Deviation** (Std Dev) shows the variability in scores. A lower standard deviation indicates more consistent quality, while a higher value suggests more fluctuation.

1. RAG's Impact on Knowledge Q&A. We analyzed 5306 valid interaction samples from the Knowledge Q&A module. Notably, *Realism* stands out with a high *Mean Score* of 90.34, coupled with the lowest standard deviation of 3.89 and the narrowest confidence interval [90.13, 90.56], indicating exceptional consistency and reliability in students' perceived realism of responses. Similarly,

Correctness (88.87), *Helpfulness* (87.87), and *Effectiveness* (85.25) are all notably higher than in the Programming Tutor module, further demonstrating the positive influence of RAG.

2. Programming Tutor (Heuristic, no RAG). In contrast, the Programming Tutor (Heuristic, no RAG) module performs well in the *Heuristic* dimension, where it achieves the highest score of 84.72. However, when compared to RAG-enhanced modules, Programming Tutor performs less consistently across other dimensions. For instance, while *Realism* (82.15) and *Effectiveness* (83.10) are still decent, they fall behind RAG's scores, highlighting that Programming Tutor is less effective in terms of providing realistic and effective guidance without RAG. Additionally, the *Correctness* dimension shows a lower score (76.23) and higher variability (Std. Dev. = 6.48), indicating more fluctuation in the quality of the responses.

To evaluate the reliability of average scores, we compute the **95% Confidence Interval**, defined by the lower and upper bounds in the last two columns of Table 9. These intervals are calculated according to Eq. (6),

$$CI_{95\%} = \bar{x} \pm z \cdot \frac{s}{\sqrt{n}}, \quad (6)$$

where $\bar{x}$ denotes the sample mean, s is the standard deviation, n is the valid sample size, and $z = 1.96$ is the critical value for 95% confidence. A narrower confidence interval implies higher statistical precision. The *Realism* dimension in the Knowledge Q&A module has the narrowest CI [90.13, 90.56], reinforcing its high score stability. This indicates that RAG's impact on *Realism* is not only positive but also very consistent. In contrast, the *Correctness* dimension in the Programming Tutor module has the widest CI [75.65, 76.80], reflecting greater instability in performance.

3. Sources of Instability in Programming Tutor and Mitigation. The lower and more variable correctness of the Programming Tutor module mainly comes from its use without RAG-based grounding. Without course-specific materials, the tutor lacks domain-specific context and often produces generic guidance that does not match the instructor's intended solution space. When we run the GLM-4-Plus model with no external context, it also fails to recover course-specific conventions, so its feedback remains vague. This weakness is most visible in algorithmic programming tasks, where the tutor must break a problem into several teaching steps and decide how to move a student from one step to the next. If the model does not see enough context, it often chooses an unhelpful path or level of detail, and its correctness becomes unstable. Based on these findings, we are iterating on the Programming Tutor along three directions: (1) we add an agent mechanism, implemented with a Model Context Protocol (MCP) (Hou et al., 2025) interface, that retrieves course-aligned background knowledge and injects it as context for each tutoring turn; (2) we fine-tune the base model on a curated dataset of course solutions and instructor feedback; and (3) we deploy a larger-capacity model in later versions of the tutor.

8.2.3. Evaluation of retrieval-augmented generation

Retrieval-Augmented Generation (RAG) enhances factual accuracy and knowledge coverage in our [learn] module. We evaluate different RAG strategies across multiple language models using the *Ragas* framework.¹⁶ For evaluation, we collect a dataset of 50 Java programming interview-style questions covering topics like *Data Structures*, *Multithreading*, and *Operators*, and incorporate an external knowledge base for LLM augmentation. We test five LLMs including GLM-4-Flash, GLM-4-Air, GLM-4-Plus, GPT-3.5-Turbo, and GPT-4 with three retrieval configurations: (1) *Without RAG*, using direct prompting; (2) *Original RAG*, with dense retrieval and lightweight reranking; and (3) *Our RAG*, which integrates multi-query rewriting, hybrid retrieval, and neural re-ranking. We evaluate using *Context Precision*, *Context Recall*, *Faithfulness*, and *Answer Relevance* metrics, employing GPT-3.5 as the judge model for segmentation, factual verification, and semantic similarity estimation. Across all models, our RAG achieved an average improvement of 2.93% in these metrics. Based on the experimental results, we draw the following conclusions, which hold significant implications for constructing RAG-based programming tutors.

1. RAG-Enhanced Retrieval Is Essential for Mid-Tier Models. As evidenced by Table 10, mid-tier models such as GPT-3.5-Turbo and GLM-4-Air show significant improvements when supported by RAG. For example, GPT-3.5-Turbo increases its Context Precision from 0.5912 to 0.8125 and boosts Answer Relevance from 0.4653 to 0.4865 under our RAG configuration. These results suggest that RAG, particularly with optimized query rewriting and reranking, enhances the relevance of responses by addressing the limited contextual grounding of mid-capacity models. In contrast, high-capacity models like GPT-4 already achieve high scores without retrieval but still benefit from RAG in areas such as Context Recall (from 0.8376 to 0.8878) and Faithfulness (from 0.9906 to 0.9948). These improvements show that even strong models gain from refined retrieval mechanisms.

2. Performance Disparities Across Models. The evaluation reveals clear performance disparities across language models. Without RAG, models like GLM-4-Flash and GPT-3.5-Turbo produce relatively weak responses, with Answer Relevance scores of 0.0433 and 0.2014, respectively. In contrast, GPT-4 performs well even without retrieval, reflecting a stronger internal knowledge base. GLM-4-Plus shows consistently high Context Precision (0.8958) and Faithfulness (0.9792) across configurations, indicating reduced dependence on external context. These variations suggest that RAG plays a compensatory role in weaker models, whereas stronger models can maintain baseline performance independently of retrieval.

3. Greater RAG Gains in Weaker Models. Weaker models show the most pronounced improvements when augmented with RAG. For example, GPT-3.5-Turbo sees the largest increase in Context Precision (+0.2213), and GLM-4-Flash shows a gain in Context Recall (+0.0677). However, the performance of these models is still influenced by the quality of the retrieval process. This highlights that retrieval must be tailored to the model's strengths to achieve consistent improvements.

¹⁶ <https://github.com/explodinggradients/ragas>

Table 10

Performance comparison of RAG strategies across models with delta improvements (Our RAG - Original RAG).

Model	Configuration	Context Precision	Faithfulness	Answer Relevancy	Context Recall
GLM-4-Flash	Original RAG	0.1458	0.9635	0.1107	0.3448
	Our RAG	0.2083	0.9635	0.0747	0.4125
	Δ (Our - Orig.)	+ 0.0625	+ 0.0000	-0.0360	+ 0.0677
	Without RAG	–	0.9397	0.0433	–
GLM-4-Air	Original RAG	0.6667	0.9730	0.3954	0.6833
	Our RAG	0.6875	0.9844	0.4077	0.7250
	Δ (Our - Orig.)	+ 0.0208	+ 0.0114	-0.0123	+ 0.0417
	Without RAG	–	0.9583	0.0433	–
GLM-4-Plus	Original RAG	0.8958	0.9792	0.5844	0.8163
	Our RAG	0.8958	0.9792	0.5979	0.8299
	Δ (Our - Orig.)	+ 0.0000	+ 0.0000	+ 0.0135	+ 0.0136
	Without RAG	–	0.9792	0.2012	–
GPT-3.5-Turbo	Original RAG	0.5912	0.9835	0.4653	0.7446
	Our RAG	0.8125	0.9844	0.4865	0.7827
	Δ (Our - Orig.)	+ 0.2213	+ 0.0009	+ 0.0212	+ 0.0381
	Without RAG	–	0.9795	0.2014	–
GPT-4	Original RAG	0.9167	0.9906	0.6628	0.8376
	Our RAG	0.9792	0.9948	0.6676	0.8878
	Δ (Our - Orig.)	+ 0.0625	+ 0.0042	+ 0.0048	+ 0.0502
	Without RAG	–	0.9635	0.5871	–

[RQ2] Insights

(1) *Improved Student Learning with RAG.* The Knowledge Q&A module with RAG significantly enhances the conversational learning experience of students by providing realistic and consistent responses. With a mean score of 90.34 in *Realism* and the lowest standard deviation (3.89), RAG ensures stable performance and fosters a reliable learning environment. When we design knowledge-based systems, it is essential to prioritize realism and minimize LLM hallucinations, ensuring authentic teaching and learning experiences.

(2) *Heuristic Module: Guiding Step-by-Step for Active Knowledge Construction.* The Programming Tutor module (Heuristic, no RAG) excels in providing structured, step-by-step guidance (score: 84.72), crucial for fostering active knowledge construction. By facilitating a guided learning process without offering direct solutions, it promotes self-regulated learning and enhances students' cognitive engagement, encouraging them to actively construct their understanding and computational thinking skills.

(3) *Promoting Knowledge Construction and Reducing Cognitive Load with SageJavon.* SageJavon enhances student learning by fostering active knowledge construction. It improves Prior Knowledge Activation (5.01 vs. 4.78), Concept Understanding (4.91 vs. 4.66), and Knowledge Transfer (5.09 vs. 4.82), helping students engage more effectively with prior knowledge. SageJavon also reduces cognitive load, particularly in Physical Demand and Frustration, offering less stressful learning compared to traditional tools.

(4) *Implications for Future AI Tutors and Programming Education.* For future intelligent tutoring systems (ITS), we suggest focusing on the development of conversational AI tutors, particularly in programming education. SageJavon supports active knowledge construction and reduces cognitive load, implying that future programming tutors should emphasize promoting active learning while reducing teacher dependency. Teachers would primarily act as content providers, while students engage with the learning process through SageJavon as a bridge. When designing AI tutors for programming, it is important to integrate educational theories that support personalized learning and scaffolding instruction.

8.3. Results on RQ3 - exercise recommendation and evaluation

To address the question, “Can exercise evaluation and recommendation based on LLMs accurately match the student’s learning level and provide appropriate suggestions?”, we integrate feedback mechanisms on the SageJavon website. These include buttons that allow students to assess whether the model’s evaluation of their code is accurate and whether the recommended exercises are aligned with their current learning levels. Besides, we analyze feedback from 45 questionnaires regarding user satisfaction with the Automated Code Evaluation and Exercise Recommendation modules.

8.3.1. Accuracy of automated code exercise evaluation (Section 6.3.4)

Feedback from the SageJavon website indicates that 82% of students consider the exercise evaluations to be accurate, confirming that the model identifies logic errors and provides actionable suggestions. In alignment with the website feedback, the questionnaire

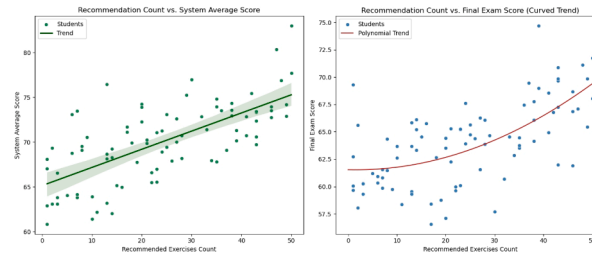


Fig. 12. Comparison between the number of recommended exercises and students' performance on both the SageJavon system and their final exam scores.

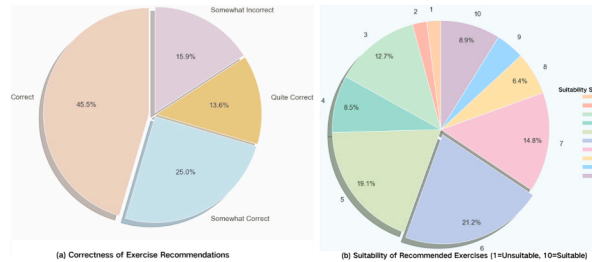


Fig. 13. Questionnaire results of automated code exercise evaluation and personalized recommendations.

results, as shown in Fig. 13(a), reveal that 45.45% of students rate the evaluations as *Correct*, 25% as *Somewhat Correct*, and 13.64% as *Quite Correct*. These results indicate the effectiveness of our proposed evaluation framework.

8.3.2. Accuracy and suitability of personalized recommendations (Section 6.4)

On the SageJavon website, 75% of students agree that the recommended exercises align well with their current learning progress in Java. Questionnaire feedback, as shown in Fig. 13(b), suggests that students who rate the exercises as suitable (scores ≥ 4) make up 72.73% of the respondents. Specifically, 22.73% of students rate the exercises as 6, 20.45% as 5, and smaller percentages rate them as 7 and above. On the other hand, 18.18% of students rate the exercises as less suitable (scores < 4), which indicates that there is still a small portion of students who feel the exercises do not adequately match their current learning level. These findings suggest that there is room for improvement in the system's recommendation model and algorithm, with potential for more precise recommendations tailored to each student's needs.

8.3.3. Correlation between recommendation count and performance

In addition to perceived usefulness, we also examined the actual learning impact of recommended exercises. Fig. 12 shows the correlation between the number of recommended exercises completed and two aspects of student performance. As the number of recommended exercises increases, students' average scores in SageJavon and their final exam scores also rise, indicating that the recommended exercises positively impact students' cognitive development and Java proficiency. We calculated the Pearson correlation coefficients to support this observation. The correlation between the number of completed recommendations and SageJavon scores was $r = 0.68$, while the correlation with final exam scores was $r = 0.62$. It indicates a moderate-to-strong relationship. Thus, SageJavon provides truly personalized recommendations for each student.

8.4. Results on RQ4 - Perspectives of teachers

We interviewed two university programming instructors to explore how they envisioned integrating SageJavon into their courses. The interviews began with their current challenges and strategies, particularly regarding students' use of LLM-based coding tools such as ChatGPT or GitHub Copilot. We then introduced SageJavon, explained its features, and shared insights from our semester-long deployment, as summarized in Section 8. Based on the qualitative analysis of the interview transcripts described below, we identified several themes regarding teachers' perspectives on SageJavon, including their likes and concerns, pedagogical and ethical considerations, and their interest in integrating the tool into their courses. Given that only two teachers participated, these findings should be viewed as exploratory and illustrative of instructor perspectives rather than statistically generalizable claims.

8.4.1. Interview method

We conducted two semi-structured interviews with university teachers who regularly teach Java or other introductory programming courses (e.g., Python). Both teachers had prior experience supervising students who used LLM-based coding tools such as ChatGPT in their coursework. In these interviews, we explored their perspectives on integrating SageJavon into their teaching practices and addressed RQ4. Each interview followed an open-ended protocol that covered four main themes: (1) teachers' experiences with

[RQ3] Insights

1. *Improve Personalized Recommendations.* AI tutors should integrate personalized models that adjust dynamically to student levels, providing accurate and context-aware feedback. As described in [Section 6.4](#), SageJavon already tailors exercise difficulty to each student's estimated mastery level. However, 18.18% of students reported that some recommendations still felt too easy or too hard, which suggests that even personalized rules can be too coarse. Future AI tutors should adapt difficulty at the level of individual students or concepts and refine recommendations based on ongoing behavioral signals, rather than relying on a fixed difficulty rule.

2. *Refine ZPD-based Personalized Recommendations.* AI tutors should integrate personalized models that adjust dynamically to student levels and provide context-aware feedback. In SageJavon, we use GIKT-predicted correctness probabilities to approximate each student's Zone of Proximal Development (ZPD) ([Shabani et al., 2010](#)) by recommending exercises with intermediate predicted success rather than relying on globally "medium" items. This design follows Vygotsky's view that learning is most effective when tasks are just beyond a learner's current independent performance and can be mastered with appropriate support. However, 18.18% of students still report that some recommended exercises feel either too easy or too hard, which suggests that a fixed probability band does not fit all learners. Future systems should refine this ZPD-based matching by adapting the target probability band per student or per concept and by incorporating additional behavioral signals (e.g., recent errors or self-reported difficulty) to further personalize recommendations.

3. *Improve Testcase-Based and LLM-Based Code Generation.* For programming AI tutors, while automated code evaluation based on test cases and syntax trees is essential, it lacks effective subjective feedback and personalized suggestions for students. Future AI tutors should focus on unbiased assessments that combine both objective and subjective evaluations. SageJavon exemplifies this approach by integrating LLM-based evaluation with traditional methods to assess their performance. However, further exploration is needed to refine the integration of these two evaluation techniques for more personalized and accurate feedback.

4. *Emphasize Data-Driven Correlation With Outcomes.* Quantitative trends show that increased engagement with recommended exercises is correlated with better system scores and higher final exam performance. Future systems should leverage these correlations to reinforce adaptive recommendation engines.

Table 11

Examples of codes in the codebook used for analyzing teacher interviews.

Code	Description	Example segment
Integration into teaching workflows	Mentions of how SageJavon could be embedded into lectures, assignments, or course structures.	"I can see using SageJavon for pre-class preparation and follow-up exercises after lectures."
Comparison with ChatGPT	Comparisons between SageJavon and general-purpose tools such as ChatGPT, including perceived benefits and risks.	"I cannot allow students to use ChatGPT for assignments, but SageJavon feels more aligned with the course."
Concerns about learning and integrity	Concerns that AI tools may reduce genuine learning or undermine assessment and academic integrity.	"If they rely too much on generic tools, they may stop thinking through the Java concepts themselves."
Desired features and teacher support	Requests for new functions or support tools, including multimodal content, dashboards, or item generation.	"A dashboard that shows which topics students struggle with would help me adjust my teaching."

existing AI tutoring or coding-assistance tools, (2) their expectations from an AI tutor like SageJavon, (3) their views on integrating SageJavon into teaching and curriculum design, and (4) suggestions for improvement and future application scenarios. We piloted the protocol with one instructor not included in the study and made minor wording adjustments to ensure clarity and relevance to teaching practice.

We conducted the interviews via video call, and each session lasted approximately 45 minutes. With participants' consent, we audio-recorded all interviews and later transcribed them verbatim. We removed identifying information (e.g., names, course titles, and institutional references) to protect participants' privacy. We followed our institution's educational research guidelines and informed participants about the study purpose, data use, and their right to withdraw at any time. The interview example is provided in [Appendix G](#).

8.4.2. Qualitative analysis of teacher interviews

To address RQ4, we conducted an iterative qualitative thematic analysis of the two interview transcripts, following established guidelines for thematic analysis ([Braun & Clarke, 2006](#)). We adopted a hybrid inductive–deductive coding strategy ([Fereday & Muir-Cochrane, 2006](#)): deductive codes were guided by the research question on teachers' views of learner-focused AI tutors, and inductive codes captured themes that emerged directly from the data. By *codebook* we refer to a list of codes, their definitions, and short example segments that we used as a shared reference when coding the interviews (see [Table 11](#)). The analysis proceeded in four phases.

1. **Familiarization and initial notes.** Two members of the research team independently read the full transcripts to become familiar with the data and to note initial impressions related to curriculum integration, pedagogical opportunities and risks, and concrete system design suggestions. Both researchers wrote brief analytic memos to record preliminary ideas.
2. **Codebook development.** The researchers collaboratively developed an initial codebook listing core codes aligned with RQ4, such as integration of SageJavon into teaching workflows, comparison with ChatGPT, concerns about student learning and academic integrity, and desired features or application scenarios for teachers. For each code, they wrote a short description and an example segment and left room for new inductive codes to be added when novel ideas appeared in the data.
3. **Coding and negotiated agreement.** Both researchers independently applied the codebook to the transcripts, using sentences or short paragraphs as the unit of analysis. When a data segment did not fit any existing code, they created a new inductive code and documented its meaning in the codebook. After this coding round, they compared their coded segments, discussed disagreements, and merged overlapping or redundant codes. Rather than relying only on a statistical measure of inter-coder reliability, we used a negotiated agreement approach and treated discrepancies as opportunities to refine code definitions until we reached consensus.
4. **Theme construction and consolidation.** We iteratively reviewed the coded data to group related codes into higher-level themes. In this phase, we repeatedly moved between the coded segments, the developing themes, and the original transcripts to ensure that the themes were well supported by the data and that negative or divergent cases were not overlooked. We kept a simple audit trail of codebook revisions and theme labels to document our analytic decisions.

We then organized the resulting themes into four result subsections that directly address RQ4: (1) *Teacher Integration and Role in Curriculum*, (2) *Comparing SageJavon with ChatGPT*, (3) *Suggestions for More Targeted Application Scenarios*, and (4) *Personalized Student Progress Visualization Dashboard*. Each subsection reflects a synthesis of multiple coded excerpts across the two interviews, rather than isolated quotes from individual teachers or anecdotal impressions.

8.4.3. Teacher integration and role in curriculum

The teachers considered SageJavon a plug-and-play solution for enhancing curriculum delivery. They noted that they could upload their course materials to the RAG backend and design course structures aligned with their existing syllabi. In this way, they envisioned SageJavon as offering a personalized learning experience that remains tightly coupled with official course content, rather than introducing generic or potentially misaligned explanations. They also highlighted that SageJavon could be integrated into different stages of a course, such as supporting pre-class preparation, in-class practice, and post-class review, thereby extending their reach beyond limited classroom time.

8.4.4. Comparing SageJavon with ChatGPT

Compared to ChatGPT, one teacher said, “I cannot allow students to use ChatGPT in my classes and assignments, but if SageJavon can do a better job in helping students learn Java, that would be great.” Another teacher noted, “Building a comprehensive learning system with multiple modules for learning, practice, assessment, and guidance, SageJavon offers more holistic support for students to gain real knowledge, compared to ChatGPT.” The teachers agreed that “SageJavon helps instructors better organize their teaching.”

These comments suggest that educators are concerned that general-purpose tools like ChatGPT might lead students to focus less on genuine learning and instead rely on them for direct answers. In contrast, SageJavon was perceived as being more tightly constrained by course materials and as structuring learning through dedicated modules for learning, practice, evaluation, and support. This design was seen as better aligned with instructional goals: SageJavon can assist students in acquiring knowledge while alleviating teaching burdens for instructors, rather than undermining assessment integrity.

8.4.5. Suggestions for more targeted application scenarios

The teachers suggested broadening SageJavon’s capabilities to include additional programming languages and multimodal content, such as instructional videos and annotated code walkthroughs. They recommended tailoring feedback to individual learning needs, providing more challenging tasks for advanced students, and offering extra help for those who are struggling. In particular, they hoped that the system could dynamically adjust the difficulty and granularity of explanations based on a student’s current performance and interaction history.

The teachers also expressed interest in a teacher-side feature powered by LLMs to automatically generate assignment and exam questions, including variations of existing questions. Such functionality could streamline assessment design, reduce repetitive work, and help instructors maintain a consistent level of difficulty and coverage across different cohorts.

8.4.6. Personalized student progress visualization dashboard

Teachers highlighted the need for a robust student monitoring dashboard that tracks progress and engagement in real time. They emphasized that such a dashboard should aggregate data from the different modules of SageJavon, including learning interactions, exercise attempts, and evaluation outcomes, to provide a holistic view of each student’s development.

This feature would help identify students who need additional support and enable teachers to tailor interventions accordingly. For example, instructors could quickly detect students who repeatedly struggle with certain Java concepts or who disengage from practice activities. The dashboard would also provide insights into common areas where students struggle, allowing teachers to apply targeted strategies, adjust teaching pace, and allocate resources more effectively at both the individual and class level.

Table 12

Comparison of SageJavon with baseline systems across key learning dimensions. ✓ indicates substantive support for a feature, ✕ indicates that the feature is not supported, and P denotes partial or limited support. Descriptive text denotes restricted or non-adaptive variants.

Dimension	LLM-based conversational tutors				Practice platforms				SageJavon
	ChatGPT	ChatGLM	CodeAid	Copilot	LeetCode	HackerRank	Exercism	Codecademy	
Course-specific knowledge	✕	✕	✓	✕	✕	✕	✕	✕	✓
RAG-enhanced knowledge retrieval	✕	✕	✕	✕	✕	✕	✕	✕	✓
Contextualized response generation	P	P	✓	P	✕	✕	✕	✕	✓
Proactive follow-up / hinting	✕	✕	✓	✕	✕	✕	✕	✕	✓
Personalized learning path	✕	✕	✕	✕	✕	✕	P	P	✓
Knowledge graph visualization	✕	✕	✕	✕	✕	✕	✕	✕	✓
Mastery-level tracking	✕	✕	✕	✕	✕	✕	P	P	✓
Personalized exercise recommendation	✕	✕	✕	✕	✕	By difficulty	By path	By course	✓
Automated code evaluation	✕	✕	✓	✓	Test cases	Test cases	Mentor review	Tests + review	✓
Student report generation	✕	✕	P	✕	✕	✕	✕	✕	✓
Teacher monitoring dashboard	✕	✕	P	✕	✕	✕	✕	✕	✓

8.5. Plug-and-play solution

We design SageJavon as a plug-and-play service for instructors and students rather than a low-level toolkit for system developers. All specialized components, including the Graph-based Interaction Knowledge Tracing (GIKT) model, the multi-stage Retrieval-Augmented Generation (RAG) pipeline, and the KTUCF-PRA recommendation algorithm, are pre-trained, configured, and deployed once on our servers. As a result, course staff do not need to collect training data, tune hyperparameters, or modify any model or pipeline configuration. In practice, “plug-and-play” means that adopting SageJavon for a new course only involves content-level operations on the RAG backend, rather than code-level or infrastructure-level changes. As detailed in Section 6.1.2, instructors can directly upload domain materials such as slides in PDF or PowerPoint format, e-books in PDF or EPUB format, structured notes in Markdown, or external URLs through a web interface. The system then automatically crawls, extracts, cleans, and indexes these resources. Once the materials are ingested, the same pre-deployed GIKT and recommendation components immediately operate over the updated knowledge base to provide learning support, practice, and personalized recommendations through a unified interface. Although pre-training GIKT and engineering the recommendation algorithms require substantial expert effort, this effort is amortized across courses and hidden from end users. Consequently, SageJavon behaves as a plug-and-play service that can be integrated into new and diverse courses with minimal setup, without retraining models and without requiring any machine learning or systems expertise from instructors or students.

9. Comparison with existing AI tutors and learning platforms

We qualitatively compare SageJavon with widely used learning tools. Baseline systems fall into two categories: (1) LLM-based conversational tutors (ChatGPT,¹⁷ ChatGLM,¹⁸ CodeAid,¹⁹ and GitHub Copilot,²⁰) and (2) practice platforms with limited or no AI support (LeetCode,²¹ HackerRank,²² Exercism,²³ and Codecademy.²⁴) Rather than benchmarking raw model performance, we examine feature- and design-level capabilities along key learning dimensions drawn from intelligent tutoring systems (ITS) research, including course alignment, knowledge retrieval, response contextualization, personalization, and performance tracking. Table 12 summarizes this comparison and indicates where SageJavon extends beyond existing solutions.

Overall, the comparison shows that SageJavon is the only system among those considered that simultaneously provides (i) course-grounded responses via RAG, (ii) mastery-level tracking with personalized exercise recommendation, (iii) multidimensional code evaluation (FUPS-Score), and (iv) teacher-facing monitoring dashboards. These capabilities operationalize pedagogical principles from ITS and mastery learning in ways that are largely absent from current AI tutors and practice platforms, thereby clarifying SageJavon’s distinctive contribution relative to existing tools.

¹⁷ <https://chatgpt.com>

¹⁸ <https://chatglm.cn>

¹⁹ <https://codeaid.io/>

²⁰ <https://github.com/features/copilot>

²¹ <https://leetcode.com>

²² <https://www.hackerrank.com>

²³ <https://exercism.org>

²⁴ <https://www.codecademy.com>

10. Educational implications and conclusion

10.1. Educational implications

The rapid evolution of LLMs and ITS has significant implications for programming education. SageJavon exemplifies how AI can create an interactive and adaptive learning experience, but its development also highlights broader challenges and opportunities for future pedagogical AI tutors. Besides the insights in [Section 8](#), We draw several educational implications.

1. Enhanced RAG-driven knowledge question answering. Integrating Retrieval-Augmented Generation (RAG) with AI tutors can enhance context-aware responses by retrieving and synthesizing relevant course materials and improving problem-solving through code-based justifications, real-world case studies, and supplementary resources. However, ensuring the trustworthiness ([Zhou et al., 2024](#)) of RAG systems remains a significant challenge. The retrieved information may sometimes be outdated, irrelevant, or misleading, while recommended resources may suffer from broken links, paywalls, or restricted access, limiting usefulness. Thus, a reliable RAG system must not only improve factual accuracy and robustness but also uphold fairness, transparency, and accountability in its information retrieval process. Additionally, educators must carefully manage privacy concerns when handling student interactions, which requires implementing secure data practices. RAG systems should prioritize high-quality, authoritative sources, ensure pedagogical value, and refine adaptive filtering mechanisms to improve response reliability and contextual relevance.

2. Beyond correctness-based grading: LLM-based task evaluations. LLM-based task evaluations can move beyond the traditional correctness-based grading, which is often binary, either correct or incorrect. Instead, they can provide a more comprehensive assessment by analyzing code structure, efficiency, maintainability, and adherence to best practices. Relying solely on test cases for grading is insufficient, as students gain more from targeted feedback and clear identification of errors. However, ensuring that LLM-based code grading aligns more closely with human evaluations remains a significant challenge, particularly in assessment criteria. Future research should refine evaluation benchmarks and develop more rigorous assessment methods. Moreover, integrating LLMs with static and dynamic code analysis tools ([Fan et al., 2023](#)) can enhance automated evaluation quality, bridging the gap between LLM-driven and human assessments while ensuring meaningful and pedagogically valuable feedback.

3. Multimodal AI tutors in Programming Education. The role of multimodal AI tutors in programming education ([Chen et al., 2024](#)) requires further exploration. These tutors can integrate multimodal capabilities at the information retrieval (RAG) and recommendation (LLM-based ITS) levels. In RAG, the retriever encodes various resources ([Barochiya et al., 2024](#)), such as text, code snippets, and instructional videos. **For students**, this process enables the LLM to generate comprehensive and contextually relevant responses, thus allowing students to access well-rounded answers. **For educators**, multimodal course materials can be easily uploaded, offering students rich learning resources with minimal effort.

Furthermore, LLM-powered Intelligent Tutoring Systems (ITS) should move beyond static text-based recommendations and provide personalized, multimodal learning experiences. They should offer resources such as PDFs, interactive coding exercises, and targeted video tutorials while dynamically adjusting recommendations based on student progress and mastery of knowledge concepts.

10.2. Conclusion

We introduce SageJavon, an AI tutor for Java programming education, based on the “Learn-Practice-Evaluate-Support” framework. Deployed in a Java Object-Oriented Programming course with 85 students, SageJavon leverages LLMs to provide personalized feedback and guidance. Through quantitative and thematic data analysis, we evaluate its effectiveness, with students expressing high satisfaction. Compared to traditional tools, SageJavon shows lower cognitive load, better knowledge transfer, and higher engagement. We explore insights for future programming AI tutors, focusing on fostering active learning through conversational modules, reducing cognitive load during learning, and incorporating enhanced RAG for more authentic, realistic responses. The theoretical and practical innovations of SageJavon demonstrate the importance of integrating educational theory into AI tutoring systems, providing essential support to students throughout their entire learning process.

Besides, we introduce a plug-and-play solution for future educational tools that can quickly and easily adapt to other programming languages. We plan to extend it to languages like Python in future courses. We also aim to integrate multimodal resources, such as videos, interactive demos or digital human. Future improvements will focus on real-time response optimization, refining knowledge tracing, and enhancing recommendation algorithms for personalized learning. These enhancements will make SageJavon more scalable, intelligent, and adaptable across various programming languages.

CRedit authorship contribution statement

Hanyu Zhao: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Yuzhuo Wu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Zhufeng Lu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Xiaohua Yu:** Writing – review & editing; **Weikai Miao:** Writing – review & editing; **Liangyu Chen:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Data availability

Data will be made available on request.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by NSFC (No. 62272416), National Education Sciences Planning Project (No. BCA240048), and the Open Project of Shanghai Driverless Train Control System Engineering Technology Research Center(SUTC-2024KT-03).

Appendix

The appendix is organized as follows: [Appendix A](#) presents the correlation analysis of cognitive load, learning outcomes, and engagement metrics. [Appendix B](#) provides additional examples of RAG-based task responses. [Appendix C](#) shows thematic scoring examples for conversational responses. [Appendix E](#) presents FUPS-Score evaluation examples for the “Rotate Array” task. [Appendix F](#) compares GPT-4 responses with and without RAG.

Appendix A. Correlation analysis of cognitive load, learning outcomes, and engagement metrics

[Fig. A.1](#) illustrates the relationships between cognitive load, learning outcomes, and engagement metrics. The values show the strength and direction of the correlations. For example, there is a strong positive correlation between Physical Demand and Temporal Demand. In contrast, Frustration negatively impacts Performance and Knowledge Transfer, suggesting that higher frustration reduces learning outcomes.

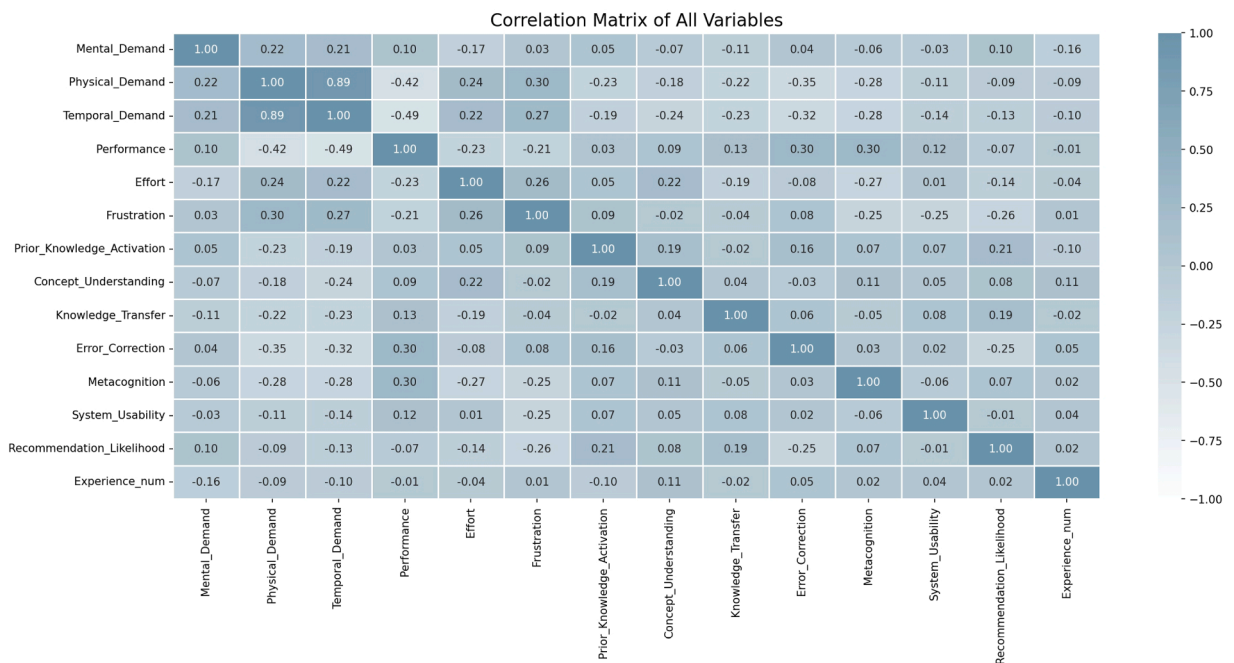


Fig. A.1. Correlation matrix displaying the relationships between cognitive load, constructivist learning outcomes, and engagement metrics, highlighting how these factors interrelate.

Appendix B. Additional examples of RAG-based task responses

To further demonstrate the versatility of our RAG system, we provide additional qualitative results for three representative task types: code generation ([Fig. B.1](#)), code bug repair ([Fig. B.2](#)). In these examples, SageJavon provides answers along with code examples and several accessible learning resource links to facilitate further learning. The learning resources are uploaded or pasted by the teacher in advance.



Fig. B.1. Example of Java code generation from textual specifications using the RAG-enhanced system.

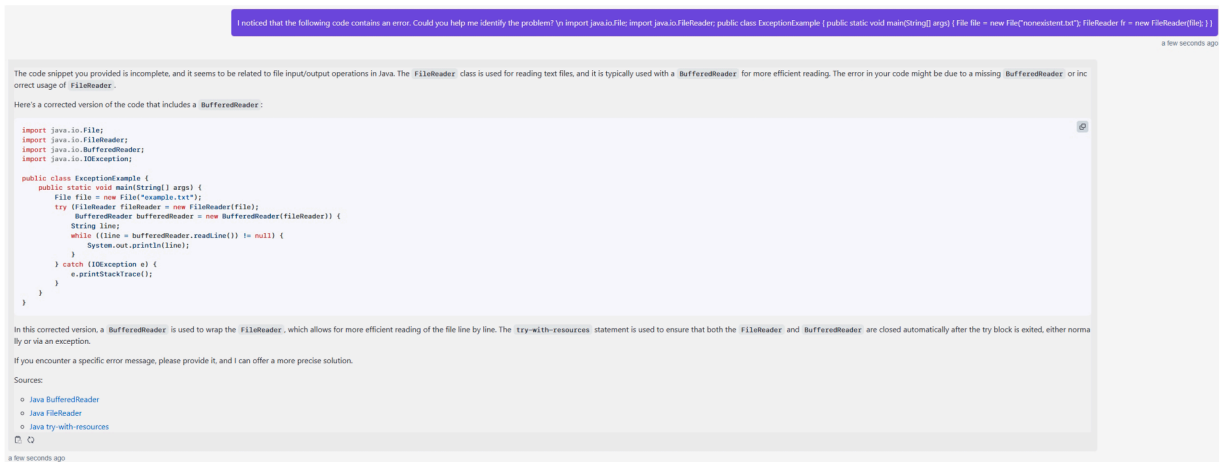


Fig. B.2. Example of automatic bug identification and correction in Java code.

Appendix C. Thematic scoring examples for conversational responses

Table C.1 provides examples for scoring the conversational-based tutor's responses based on the thematic analysis dimensions: Correctness (C), Helpfulness (H), Realism (RAG) (R), and Avoidance of Direct Code Solutions (Heuristic) (A). Each response is evaluated across these dimensions, with scores assigned based on the quality of the response in each area.

Table C.1

Examples of scoring based on thematic analysis dimensions for conversational-based tutor responses. Each response is scored on multiple dimensions, and the pass status is determined based on these scores. The average score across all dimensions is also provided.

Category	Query	Response	Dimension	Score	Pass	Average Score
Knowledge Q&A Module	“What is inheritance in Java?”	“Inheritance in Java allows one class to inherit the properties and methods of another class. This helps in achieving reusability and method overriding, which are core concepts in object-oriented programming. For more details, please refer to this link: [A reachable Link about inheritance].”	Correctness	85	✓	85>80 ✓
			Helpfulness	80	✓	
			Realism	85	✓	
			Effectiveness	90	✓	
Programming Tutor Module	“How do I implement a bubble sort in Java?”	“Let’s start by thinking about how the bubble sort algorithm works. It compares two adjacent elements and swaps them if they are in the wrong order. Here’s a framework for the algorithm in Java: <ddq>“Java for (int i = 0; i < arr.length - 1; i++) for (int j = 0; j < arr.length - i - 1; j++) if (arr[j] > arr[j + 1]) // swap elements ’</ddq>’. Now, can you try implementing this in your own code, filling in the swap logic?”	Correctness	88	✓	88>80 ✓
			Helpfulness	92	✓	
			Avoidance of Direct Code Solutions	85	✓	
Knowledge Q&A Module (Low Score)	“What is polymorphism in Java?”	“Polymorphism allows objects of different types to be treated as objects of a common supertype. This allows methods to be written that can operate on objects of different classes. However, polymorphism also depends on other factors like method overloading, which is not quite the same thing. For more information, you can refer to this resource: [An unreachable Link about inheritance].”	Correctness	70	×	62<80 ×
			Helpfulness	60	×	
			Realism	60	✓	
			Effectiveness	60	✓	
Programming Tutor Module (Low Score)	“How do I sort an array in Java?”	“You can sort an array by using the Arrays.sort() method, which is part of the Java Collections framework. However, it’s a good idea to understand how sorting works manually. Can you try to implement a basic sorting algorithm, like bubble sort or selection sort, yourself? ”	Correctness	65	×	66<80 ×
			Helpfulness	55	×	
			Avoidance of Direct Code Solutions	80	✓	

Appendix D. Sensitivity analysis on the similarity threshold θ

To assess the effect of the similarity threshold θ on peer coverage and recommendation quality, we conducted a sensitivity analysis on a held-out subset of the interaction logs. For each $\theta \in \{0.90, 0.92, 0.95\}$, we recomputed the similar-student sets S_i , rebuilt the candidate exercise sets E_i^{final} , and evaluated the resulting recommendations using an offline Hit@10 metric based on held-out student responses. We also measured the median size of S_i and the proportion of students with at least one peer in S_i . As θ decreases from 0.95 to 0.90, both the median peer-set size $|S_i|$ and the coverage increase, confirming that lower thresholds group more students into each neighborhood. However, the offline Hit@10 scores remain in a narrow range between 0.41 and 0.42 and do not exhibit a clear monotonic trend. In particular, $\theta = 0.95$ yields smaller but more homogeneous peer sets while achieving a Hit@10 comparable to that of lower thresholds. Taken together, these results (Table D.1) suggest that $\theta = 0.95$ provides a reasonable trade-off between peer-set homogeneity and practical coverage in our recommendation setting.

Table D.1

Effect of the similarity threshold θ on peer coverage and offline recommendation quality.

θ	Median $ S_i $	% students with $ S_i > 0$	Hit@10
0.90	14	97%	0.41
0.92	9	93%	0.42
0.95	4	78%	0.41

Appendix E. FUPS-score evaluation examples

To enhance the transparency, interpretability, and reproducibility of the proposed *FUPS-Score* evaluation framework, we present a comprehensive set of annotated examples based on a representative programming task, “*Rotate Array*”. The task is assessed across three key dimensions: *Functional Correctness* (C_f), *Usefulness* (C_u), and *Programming Style* (C_s), in accordance with the scoring rubrics defined in Fig. 8. The evaluation results for five representative student solutions are summarized in Table E.1, where each solution is assessed across all three dimensions. For each dimension, we report the assigned score (0–4), a concise explanation of the rationale, and the LLM-generated suggestion for improvement. Additionally, Table E.2 provides the full Java implementations of all five code snippets evaluated in the scoring section, referenced via labels (e.g., Listing 1) in the scoring table. Each row in the scoring table

Table E.1

FUPS-Score Evaluation for Each Code Snippet with Detailed LLM Feedback and Example Fixes.

Code	Dimension	Score	Explanation	LLM Feedback and Example Suggestion
Listing 1	FUNCTIONAL CORRECTNESS	0	This code does not solve the problem. It simply reassigns the ‘nums’ array to a new array, which doesn’t address the rotation requirement.	LLM Suggestion: The student misunderstood the task. You should not create a new array unless necessary. Try shifting elements manually: <code>int temp = nums[n - 1]; for (int j = n - 1; j > 0; j--) nums[j] = nums[j - 1]; nums[0] = temp;</code>
	USEFULNESS	0	Since the original data is discarded, the solution has no practical value in any application setting.	
	PROGRAMMING STYLE	0	Code is compact, unindented, and uses vague variable names, which makes it unreadable.	
Listing 2	FUNCTIONAL CORRECTNESS	1	The code compiles and runs, but it inefficiently performs the rotation in $O(nk)$ time. The code is correct but not optimal for large arrays.	LLM Suggestion: While the logic is correct, it’s inefficient. Replace the repeated rotation with a more scalable in-place reversal: <code>reverse(nums, 0, n - 1); reverse(nums, 0, k - 1); reverse(nums, k, n - 1);</code>
	USEFULNESS	1	This solution produces a result but does not operate efficiently on larger inputs.	
	PROGRAMMING STYLE	1	Code is tightly packed without any explanation, reducing clarity and maintainability.	
Listing 3	FUNCTIONAL CORRECTNESS	2	The result is logically correct and efficient in time, but it consumes extra memory due to the temporary array.	LLM Suggestion: Avoid extra memory by doing in-place reversal instead. You can achieve the same result without using a temporary array: <code>reverse(nums, 0, n - 1); reverse(nums, 0, k - 1); reverse(nums, k, n - 1);</code>
	USEFULNESS	2	The code is correct and reusable but not optimal for memory-constrained scenarios.	
	PROGRAMMING STYLE	2	Code structure is basic, with some comments, but lacks abstraction or helper methods.	
Listing 4	FUNCTIONAL CORRECTNESS	3	This version rotates the array using the three-step reversal method. It is both correct and efficient.	LLM Suggestion: The code is strong but would benefit from better documentation for maintainability: <code>// Step 1: Reverse entire array // Step 2: Reverse first k elements // Step 3: Reverse the rest</code>
	USEFULNESS	3	The approach is clean and efficient, but adding comments would improve collaboration.	
	PROGRAMMING STYLE	3	Structure is good, but documentation is limited. JavaDoc for the helper method is recommended.	
Listing 5	FUNCTIONAL CORRECTNESS	4	A fully correct implementation with clean helper methods and documentation.	LLM Suggestion: No improvements needed. This implementation is well-documented and suitable for production environments.
	USEFULNESS	4	Readable, reusable, and efficient for large inputs.	
	PROGRAMMING STYLE	4	Fully structured with clean JavaDoc and abstraction.	

contains the following information: (1) **Code Snippet Reference**, which refers to the evaluated code listing (e.g., Listing 2); (2) **Dimension**, specifying one of the three FUPS dimensions being evaluated; (3) **Score**, a value from 0 to 4 indicating the performance level; (4) **Explanation**, a brief rationale justifying the score; and (5) **Suggestion**, which provides specific feedback intended to help improve the submission.

Table E.2

Code Listings for Scoring Examples.

Listing 1: Score 0 <pre>public void rotate(int[] nums, int k) { nums = new int[nums.length]; // reset the array }</pre>	Listing 2: Score 1 <pre>public void rotate(int[] nums, int k) { int n = nums.length; k = k % n; for (int i = 0; i < k; i++) { int temp = nums[n - 1]; for (int j = n - 1; j > 0; j--) { nums[j] = nums[j - 1]; } nums[0] = temp; } }</pre>	Listing 3: Score 2 <pre>public void rotate(int[] nums, int k) { int n = nums.length; k = k % n; int[] temp = new int[k]; for (int i = 0; i < k; i++) { temp[i] = nums[n - k + i]; } for (int i = n - 1; i >= k; i--) { nums[i] = nums[i - k]; } for (int i = 0; i < k; i++) { nums[i] = temp[i]; } }</pre>
Listing 4: Score 3 <pre>public void rotate(int[] nums, int k) { int n = nums.length; k = k % n; reverse(nums, 0, n - 1); reverse(nums, 0, k - 1); reverse(nums, k, n - 1); } private void reverse(int[] nums, int start, int end) { while (start < end) { int temp = nums[start]; nums[start] = nums[end]; nums[end] = temp; start++; end--; } }</pre>	Listing 5: Score 4 <pre>public void rotate(int[] nums, int k) { int n = nums.length; k = k % n; reverse(nums, 0, n - 1); reverse(nums, 0, k - 1); reverse(nums, k, n - 1); } /** * Reverses the elements between * the start and end indices in * the array. * This method is called multiple * times to achieve the desired * rotation. * * @param nums The array to be * rotated. * @param start The starting index * of the segment to reverse. * @param end The ending index of * the segment to reverse. */ private void reverse(int[] nums, int start, int end) { while (start < end) { int temp = nums[start]; nums[start] = nums[end]; nums[end] = temp; start++; end--; } }</pre>	

Appendix F. RAG evaluation example: Prompted response comparison and scoring

To illustrate the evaluation process of our RAG framework, we present a detailed example comparing GPT-4's responses under two retrieval configurations: **Our RAG** and **Original RAG**. As shown in Table F.1, both systems are prompted with the same question regarding the difference between `==` and `equals` in Java.

Table F.1

Comparison of GPT-4 responses with and without RAG for the question: "What is the difference between `==` and `equals` in Java?".

System	Response
GPT-4 (Our RAG)	1. <code>==</code> operator: Compares whether two references point to the same object in memory. <pre>String a = new String("hello"); String b = new String("hello"); System.out.println(a == b); // false</pre> 2. <code>equals</code> method: Defined in <code>Object</code>, but commonly overridden to compare contents. <pre>System.out.println(a.equals(b)); // true</pre> 3. Usage difference: <code>==</code> is for primitive or reference identity; <code>equals</code> for content equality. 4. Guidelines for overriding <code>equals</code>: • Reflexivity: <code>x.equals(x)</code> is true. • Symmetry: <code>x.equals(y) == y.equals(x)</code> • Transitivity: if <code>x.equals(y)</code> and <code>y.equals(z)</code>, then <code>x.equals(z)</code> • Consistency: repeated calls yield same result. • Null-safety: <code>x.equals(null)</code> is false. Reference: equals/index.html">https://liaoخuefeng.com/books/java/collection>equals/index.html
GPT-4 (Original RAG)	1. <code>==</code> operator: • For primitives: compares values. • For objects: compares memory addresses. 2. <code>equals</code> method: By default same as <code>==</code>, but usually overridden to compare content. Example: <pre>public boolean equals(Object o) { if (o instanceof Person p) { return name.equals(p.name) && age == p.age; } return false; }</pre> 3. Overriding rules: Reflexivity, Symmetry, Transitivity, Consistency, Null-safety. 4. Use Case: Overriding <code>equals</code> is critical for collections like <code>List.contains()</code>.

System	Context Precision	Faithfulness	Answer Relevance	Context Recall
GPT-4 (Our RAG)	0.9792	0.9948	0.7186	0.8878
GPT-4 (Original RAG)	0.9167	0.9906	0.6609	0.8376

Appendix G. Interview protocol for RQ4 (perspectives of teachers)

We present the semi-structured interview protocol used to address RQ4. Tables G.1 and G.2 summarize the main themes and example questions. During the interviews, the interviewer used these questions as flexible prompts rather than a fixed script and asked follow-up questions as needed.

Table G.1

Summary of the semi-structured interview protocol for RQ4 (Part 1).

Theme	Example questions / prompts
Background and teaching context	• Could you briefly describe the courses you currently teach (e.g., programming languages, level, class size)? • What is your typical approach to teaching programming (e.g., lectures, labs, projects, online activities)?
Experiences with existing AI tools	• In your teaching, have you seen students use tools such as ChatGPT, GitHub Copilot, or other AI-based coding assistants? If so, how do they use them? • What benefits and challenges have you observed when students use these tools (e.g., learning support, academic integrity, over-reliance)? • How have you responded to or managed students' use of these tools in your courses?
First impressions of SageJavon	• Based on the description and examples, what are your first impressions of SageJavon? • Are there aspects of SageJavon that you find particularly promising or concerning for your teaching?
Expectations for an AI tutor	• In your view, what would an ideal AI tutor for programming courses be able to do for students? • To what extent do you think SageJavon aligns with this ideal? In which ways does it meet or fall short of your expectations?

Table G.2

Summary of the semi-structured interview protocol for RQ4 (Part 2).

Theme	Example questions / prompts
Integration into teaching and curriculum	• How could you imagine integrating SageJavon into your existing courses (e.g., pre-class preparation, in-class activities, homework support, review)? • In which parts of the course do you think SageJavon would be most helpful or appropriate? • How might SageJavon influence the way you design assignments, exercises, or assessments?
Pedagogical and ethical considerations	• How do you think a tutor like SageJavon could influence students' learning strategies and engagement with programming concepts? • How do you balance support from AI tutors with students' need for genuine practice and understanding? • Are there any ethical or fairness issues you would consider when adopting an AI tutor like SageJavon?
Suggestions for improvement and teacher support	• What additional features or capabilities would make SageJavon more useful for you and your students (e.g., new languages, multimodal content, different exercise types)? • How could SageJavon better adapt to students at different levels (e.g., beginners vs. advanced students)? • What kinds of teacher-facing tools or dashboards would you find most valuable (e.g., progress summaries, alerts about struggling students, topic-level analytics)?
Closing questions	• Overall, how likely would you be to adopt a system like SageJavon in your future courses, and under what conditions? • Is there anything else about AI tutors or SageJavon that you think is important but that we have not discussed?

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altschmidt, J., Altman, S., Anadkat, S. et al. (2023). Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Adeoye-Olatunde, O. A., & Olenik, N. L. (2021). Research and scholarly methods: Semi-structured interviews. *Journal of the American College of Clinical Pharmacy*, 4(10), 1358–1367.
- Algarni, A. (2016). Data mining in education. *International Journal of Advanced Computer Science and Applications*, 7(6), 456–461. <https://doi.org/10.14569/IJACSA.2016.070659>
- Bada, S. O., & Olusegun, S. (2015). Constructivism learning theory: A paradigm for teaching and learning. *Journal of Research & Method in Education*, 5(6), 66–70.
- Barochiya, M., Makhijani, P., Patel, H. N., Goel, P., & Patel, B. (2024). Evaluating RAG pipeline in multimodal LLM-based question answering systems. In *Proceedings of the 3rd international conference on automation, computing and renewable systems* (pp. 69–75).
- Bloom, B. S. (1968). Learning for mastery. *Evaluation quarterly*, 1(2), 1–12.
- Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), 77–101.
- Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., & Chen, W. (2022). Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*.
- Chen, Y., Xiao, S., Song, Y., Li, Z., Sun, L., & Chen, L. (2024). Mindscratch: A visual programming support tool for classroom learning based on multimodal generative AI. *arXiv preprint arXiv:2412.09001*.
- Conati, C. (2009). Intelligent tutoring systems: New challenges and directions. In *Proceedings of the 21st international joint conference on artificial intelligence* (pp. 2–7).
- Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4, 253–278.
- Denny, P., Kumar, V., & Giacaman, N. (2023). Conversing with copilot: Exploring prompt engineering for solving cs1 problems using natural language. In *Proceedings of the 54th ACM technical symposium on computer science education* (pp. 1136–1142).
- Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., Hu, S., Chen, Y., Chan, C.-M., Chen, W. et al. (2023). Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3), 220–235.
- Dong, Q., Li, L., Dai, D., Zheng, C., Wu, Z., Chang, B., Sun, X., Xu, J., & Sui, Z. (2022). A survey on in-context learning. *arXiv preprint arXiv:2301.00234*.
- Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., Peng, J., Sha, C., Peng, X., & Lou, Y. (2023). Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *arXiv preprint arXiv:2308.01861*.
- Eilermann, S., Wehmeier, L., Niggemann, O., & Deuter, A. (2023). Kiaaa: An ai assistant for teaching programming in the field of automation. In *Proceedings of the 21st international conference on industrial informatics* (pp. 1–7).
- Fan, G., Xie, X., Zheng, X., Liang, Y., & Di, P. (2023). Static code analysis in the AI era: An in-depth exploration of the concept, function, and potential of intelligent code analysis agents. *arXiv preprint arXiv:2310.08837*.

- Fereday, J., & Muir-Cochrane, E. (2006). Demonstrating rigor using thematic analysis: A hybrid approach of inductive and deductive coding and theme development. *International Journal of Qualitative Methods*, 5(1), 80–92.
- Forehand, M. (2010). Bloom's taxonomy. *Emerging Perspectives on Learning, Teaching, and Technology*, 41(4), 47–56.
- Frankford, E., Sauerwein, C., Bassner, P., Krusche, S., & Breu, R. (2024). Ai-tutoring in software engineering education. In *Proceedings of the 46th international conference on software engineering: Software engineering education and training* (pp. 309–319).
- GLM, T., Zeng, A., Xu, B., Wang, B., Zhang, C., Yin, D., Zhang, D., Rojas, D., Feng, G., Zhao, H. et al. (2024). Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793*.
- Grigorov, I., Bayliss-Brown, G., Murphy, D., Thøgersen, T. L., & Mariani, P. (2017). Passive vs active knowledge transfer: Boosting grant proposal impact. In *European geosciences union general assembly 2017* (p. 1). (vol. 19).
- Groothuysen, S., van den Beemt, A., Remmers, J. C., & van Meeuwen, L. W. (2024). Ai chatbots in programming education: Students' use in a scientific computing course and consequences for learning. *Computers and Education: Artificial Intelligence*, 7, 100290.
- Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y. K. et al. (2024a). Deepseek-coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.
- Guo, S., Liao, L., Zhang, J., Li, C., & Chen, H. (2024b). Pcpqr: Proactive conversational question planning with reflection. *arXiv preprint arXiv:2410.01363*.
- van Haaster, K., & Hagan, D. (2004). Teaching and learning with bluej: An evaluation of a pedagogical tool. *Issues in Informing Science and Information Technology*, 1, 455–470. <https://api.semanticscholar.org/CorpusID:266021819>.
- Halford, G. S., Cowan, N., & Andrews, G. (2007). Separating cognitive capacity from knowledge: A new hypothesis. *Trends in Cognitive Sciences*, 11(6), 236–242.
- Hanks, B., Fitzgerald, S., Mccauley, R., Murphy, L., & Zander, C. (2011). Pair programming in education: A literature review. *Computer Science Education*, 21(2), 135–173. <https://doi.org/10.1080/08993408.2011.579808>
- Hein, G. E. (1991). Constructivist learning theory. In *Proceedings of the international committee of museum educators conference* (pp. 1–10).
- Hou, X., Zhao, Y., Wang, S., & Wang, H. (2025). Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*.
- Hu, Y., Zhou, Q., Chen, Q., Li, X., Liu, L., Zhang, D., Kachroo, A., Oz, T., & Tripp, O. (2025). Qualityflow: An agentic workflow for program synthesis controlled by LLM quality checks. *arXiv preprint arXiv:2501.17167*.
- Huang, D., Bu, Q., Zhang, J. M., Luck, M., & Cui, H. (2023). Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010*.
- Kaila, E., Rajala, T., Laakso, M.-J., Lindén, R., Kurvinen, E., & Salakoski, T. (2014). Utilizing an exercise-based learning tool effectively in computer science courses. *Olympiads in Informatics*, 8, 93.
- Kazemitabbar, M., Ye, R., Wang, X., Henley, A. Z., Denny, P., Craig, M., & Grossman, T. (2024). Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs. In *Proceedings of the CHI conference on human factors in computing systems* (pp. 1–20).
- Kim, S., Ha, J., Lee, H., Park, S., & Cho, S. (2026). Human-guided collective LLM intelligence for strategic planning via two-stage information retrieval. *Information Processing & Management*, 63(1), 104288. <https://doi.org/10.1016/J.IPM.2025.104288>
- Kirschner, P. A. (2002). Cognitive load theory: Implications of cognitive load theory on the design of learning. *Learning and Instruction*, 12(1), 1–10. [https://doi.org/10.1016/S0959-4752\(01\)00014-7](https://doi.org/10.1016/S0959-4752(01)00014-7)
- Laakso, M.-J., Kaila, E., & Rajala, T. (2018). VILLE—collaborative education tool: Designing and utilizing an exercise-based learning environment. *Education and Information Technologies*, 23, 1655–1676.
- Le, N.-T., Strickroth, S., Gross, S., & Pinkwart, N. (2013). A review of AI-supported tutoring approaches for learning programming. *Proceedings of Advanced Computational Methods for Knowledge Engineering*, (pp. 267–279).
- Lewis, M. (2019). Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th international conference on neural information processing systems* (pp. 9459–9474).
- Li, L., Geng, S., Li, Z., He, Y., Yu, H., Hua, Z., Ning, G., Wang, S., Xie, T., & Yang, H. (2024). Infibench: Evaluating the question-answering capabilities of code large language models. In *Proceedings of the 37th International Conference on Advances on Neural Information Processing Systems*, (pp. 128668–128698).
- Li, Q., Li, Y., Zhang, S., Zhou, X., & Pan, Z. (2025). A theoretical framework for human-centered intelligent information services: A systematic review. *Information Processing & Management*, 62(1), 103891. <https://doi.org/10.1016/J.IPM.2024.103891>
- Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J. et al. (2023). Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.
- Liebenow, L. W., Schmidt, F. T. C., Meyer, J., & Fleckenstein, J. (2025). Self-assessment accuracy in the age of artificial intelligence: Differential effects of LLM-generated feedback. *Computers & Education*, 237, 105385. <https://doi.org/10.1016/J.COMPEDU.2025.105385>
- Lin, F., Kim, D. J. et al. (2024). When llm-based code generation meets the software development process. *arXiv preprint arXiv:2403.15852*.
- Liu, M., Zhang, L. J., & Biebricher, C. (2024). Investigating students cognitive processes in generative AI-assisted digital multimodal composing and traditional writing. *Computers & Education*, 211, 104977. <https://www.sciencedirect.com/science/article/pii/S0360131523002543>. <https://doi.org/10.1016/j.compedu.2023.104977>
- Lu, J. J., & Fletcher, G. H. L. (2009). Thinking about computational thinking. In *Proceedings of the 40th ACM technical symposium on computer science education* (pp. 260–264).
- Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D. et al. (2021). Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664*.
- Luo, Z., Xu, C., Zhao, P., Sun, Q., Geng, X., Hu, W., Tao, C., Ma, J., Lin, Q., & Jiang, D. (2023). Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*.
- Ma, I., Krone-Martins, A., & Videira Lopes, C. (2024). Integrating ai tutors in a programming course. In *Proceedings of the 2024 on ACM virtual global computing education conference* (pp. 130–136).
- McCauley, R., Fitzgerald, S., Lewandowski, G., Murphy, L., Simon, B., Thomas, L., & Zander, C. (2008). Debugging: A review of the literature from an educational perspective. *Computer Science Education*, 18(2), 67–92.
- Mota Vieira, A. V., & Magalhães Netto, J. F. (2023). Adaptive learning in programming education: A systematic mapping of the literature. In *Proceedings of the 2023 IEEE frontiers in education conference (FIE)* (pp. 1–7). <https://doi.org/10.1109/FIES8773.2023.10342933>
- Naik, D., Naik, I., & Naik, N. (2024). Decoder-only transformers: The brains behind generative AI, large language models and large multimodal models. In *Proceedings of the international conference on computing, communication, cybersecurity and AI* (pp. 315–331).
- Newstrom, J. W., & Lengnick-Hall, M. L. (1991). One size does not fit all. *Training & Development*, 45(6), 43–48.
- Nutalapati, H., Velmurugan, S., & Tiglaio, N. M. (2024). Coding buddy: An adaptive AI-powered platform for personalized learning. In *Proceedings of the 2024 international symposium on networks, computers and communications* (pp. 1–6). <https://doi.org/10.1109/ISNCC62547.2024.10759044>
- Nwana, H. S. (1990). Intelligent tutoring systems: An overview. *Artificial Intelligence Review*, 4, 251–277. <https://api.semanticscholar.org/CorpusID:206771063>.
- Okonkwo, C. W., & Ade-Ibijola, A. (2020). Python-bot: A chatbot for teaching python programming. *Engineering Letters*, 29(1), 25–34.
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the association for computational linguistics* (pp. 311–318).
- Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L., & Sohl-Dickstein, J. (2015). Deep knowledge tracing. In *Proceedings of the 29th international conference on neural information processing systems - volume 1* (p. 505–513).
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67.
- Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., & Ma, S. (2020). Codebleu: A method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*.

- Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J. et al. (2023). Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.
- Schildkamp, K., van der Kleij, F. M., Heitink, M. C., Kippers, W. B., & Veldkamp, B. P. (2020). Formative assessment: A systematic review of critical teacher prerequisites for classroom practice. *International Journal of Educational Research*, 103, 101602.
- Setiawan, A., Irfanuddin, M. F., Adhisa, R. R. et al. (2024). Effectiveness of AI-based chatbots as virtual tutors in basic programming education. *Jurnal Teknologi Informasi dan Pendidikan*, 17(2), 482–492.
- Shabani, K., Khatib, M., & Ebadi, S. (2010). Vygotsky's zone of proximal development: instructional implications and teachers' professional development. *English Language Teaching*, 3(4), 237–248.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Proceedings of the 31st international conference on neural information processing systems* (p. 6000–6010).
- Wang, Y., Le, H., Gotmare, A. D., Bui, N. D. Q., Li, J., & Hoi, S. C. H. (2023). Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922*.
- Xuan, J., Martinez, M., Demarco, F., Clement, M., Marcote, S. L., Durieux, T., Le Berre, D., & Monperrus, M. (2016). Nopol: Automatic repair of conditional statement bugs in java programs. *IEEE Transactions on Software Engineering*, 43(1), 34–55.
- Yang, B., Tian, H., Pian, W., Yu, H., Wang, H., Klein, J., Bissyandé, T. F., & Jin, S. (2024). Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis* (pp. 882–894).
- Yang, Y., Shen, J., Qu, Y., Liu, Y., Wang, K., Zhu, Y., Zhang, W., & Yu, Y. (2020). Gikt: A graph-based interaction model for knowledge tracing. *ArXiv, abs/2009.05991*. <https://api.semanticscholar.org/CorpusID:221654987>.
- Yin, P., Deng, B., Chen, E., Vasilescu, B., & Neubig, G. (2018). Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th international conference on mining software repositories* (pp. 476–486).
- Zhang, S., Dong, L., Li, X., Zhang, S., Sun, X., Wang, S., Li, J., Hu, R., Zhang, T., Wu, F. et al. (2023). Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792*.
- Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., Yang, L., Zhang, W., & Cui, B. (2024). Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473*.
- Zhou, S., Alon, U., Agarwal, S., & Neubig, G. (2023). Codebertscore: Evaluating code generation with pretrained models of code. *arXiv preprint arXiv:2302.05527*.
- Zhou, Y., Liu, Y., Li, X., Jin, J., Qian, H., Liu, Z., Li, C., Dou, Z., Ho, T.-Y., & Yu, P. S. (2024). Trustworthiness in retrieval-augmented generation systems: A survey. *arXiv preprint arXiv:2409.10102*.
- Zhu, J., Miao, Y., Xu, T., Zhu, J., & Sun, X. (2024). On the effectiveness of large language models in statement-level code summarization. In *Proceedings of the 24th international conference on software quality, reliability and security* (pp. 216–227).
- Zhuo, T. Y. (2024). ICE-score: Instructing large language models to evaluate code. In *Findings of the association for computational linguistics: EACL* (pp. 2232–2242). <https://aclanthology.org/2024.findings-eacl.148/>.